

The Alberta Buck - Notes Flow (v1.1)

How One BUCK Transfer Becomes N Unlinkable Notes

Perry Kundert

2026-06-10



This document is the *narrative and executable* companion to alberta-buck-notes.org (architectural overview; together they are the two living Notes references) and alberta-buck-ethereum.org (Solidity surface). It walks one concrete example – Alice mints a 1,200-BUCK transfer into six notes of denominations [1000, 100, 25, 25, 25, 25] – end-to-end through the **Phase-7-bis SNARK-verified batch-mint pipeline**, and then walks the *inverse* flows: A2 deposit (Bob redeems his identity-targeted note via the deposit-coupling sigma + membership proof) and B1 deposit (Carol redeems a bearer note while the issuer learns who cashed it). Every actor, every artifact, and every check is named.

The mint pipeline uses SNARK-verified batch minting: the circuit takes `oldRoot` as a public input, folds N commitments into the rolling tree using witness sibling paths, and emits a SNARK-attested `newRoot` for the contract to record. On-chain mint cost is essentially constant in $N - \sim 380K$ plus ~ 500 gas per leaf of calldata. The full design memo, including RICS counts, gas projections, and a per-attack-vector hostile-Alice analysis, is alberta-buck-notes-rollup-mint.org.

The *identity-M* spend model supersedes the prior Phase-7/8 account-pinned circuits. Under the identity-M model the authority axis is the KYC-bound Identity point M, not the EOA. Every spend is one gadget – an EIP-196 Okamoto sigma producing a committed point *P*, plus a Poseidon-Merkle membership proof *bound to that same P* (`identity_membership_g1tie.circom`, $\sim 350K$ gas on chain) – instantiated three ways: `Notes.spendCoupledA2` (addressed, private issuer, `verifyDepositCoupling`), `spendCoupledA1` (addressed, public issuer; the recipient's own Identity is committed), and `spendCoupledB1` (bearer, public issuer, `verifyDepositorBinding`). This *retires* the earlier account-pinned `spendACP` / CP-DLEQ path and the unbound generic `Notes.spend`. The membership closes the collusion gap: a note whose committed Identity decrypts to a non-member reverts at spend. All three are wired and tested end to end (`alberta_buck.wallet.unilateral_a2`, `.unilateral_a1`, `.b1_binding`; `IdentityRegistry.verifyDepositCoupling`, `.verifyDepositorBinding`;

the G1-tie membership verifier; the on-chain incremental `identityRoot`). A fourth gate completes the addressed spends: the note $\leftrightarrow eEnc$ re-encryption tie (`circuits/note_binding.circom` behind `INoteBindingVerifier`) proves the deposit-coupling ciphertext re-encrypts, under M_{rec} , the ciphertext *the spent note committed in its idHash* – enforcing addressed-binding (a stolen opening cannot be redeemed with a self-addressed *eEnc*) and closing A2 collusion against ciphertext substitution (*Proofs*, Theorem 12). Only a production Powers-of-Tau ceremony remains before deployment.

The cryptographic walkthroughs in this document are *executable*: the Python blocks run against the shipped `alberta_buck.wallet` modules in one seeded, deterministic session, and their printed output is captured below each block.

The aim is to show, by walking the path, that the system delivers five concrete guarantees:

1. **Atomic conservation** – if the transaction succeeds, the BUCK balance leaves Alice exactly once and matches the public total of the new notes. If the proof fails, no BUCK moves and no commitments are appended.
2. **Issuer-bound, recipient-blinded artifacts** – the public commitments name the issuer (`msg.sender`) but reveal nothing about per-note values or recipients.
3. **Per-note disclosure to recipients only** – each recipient receives an *opening* off-chain that lets them reconstruct exactly their own note’s value and the issuer’s verifiable identity, plus the fact that N siblings exist whose hidden values sum to `totalFace` – and nothing more.
4. **Offline receipt reconstruction** – given only the mint transaction ID, anyone holding an opening can independently verify the note against the chain, with no help from Alice or any indexer.
5. **Adversarial-prover soundness** – even if Alice runs a maliciously patched EVM and an arbitrarily hostile witness generator, the honest validators executing the deployed `Notes.sol` and `MintGroth16Verifier.sol` reject every batch that does not satisfy the mint relation. The on-chain truth is what governs.

The remaining caveat (the pinned- N dispatch set under `mint_batch.circom`) is addressed at the end and cross-referenced into the proofs and implementation docs. The mint-circuit field-overflow concern from earlier drafts has been closed by the `Num2Bits(128)` range bounds in `mint_batch.circom` and the matching bounds in `spend.circom`. The identity-binding gap that earlier drafts flagged as deferred is now closed: each spend’s deposit-coupling / depositor-binding sigma proves the spender is the Identity the note names, and the membership proof – bound to the sigma’s own committed point P – proves that Identity is a registered member of the on-chain accumulator.

Contents

1	Cast of Characters	5
2	The Promise	5
3	What’s Inside a Note (Conceptually)	6
4	The Mint, Step by Step	6
4.1	Step 0: Pre-flight (Off-Chain, In Alice’s Wallet)	6
4.2	Step 1: Build Commitments	7
4.3	Step 2: Generate the Batch-Mint Groth16 Proof	7
4.4	Step 3: Submit <code>Notes.mint</code> On Chain	8
4.5	Step 4: Off-Chain Note Delivery	10
4.6	Step 5: Recipient Verification (Offline, Independent)	11
5	Why Alice Cannot Cheat (Even With a Hostile EVM and Prover)	12
5.1	Pillar 1: The On-Chain Verifier Is Fixed Code	12
5.2	Pillar 2: BUCK Escrow Is Inside the Same Atomic Call	12
5.3	Pillar 3: Public Inputs Are Sealed Into the Proof	13
5.4	Putting the Pillars Together: The Adversarial Walk	13
6	Why Outside Observers Cannot Link	14
6.1	Spend-Time Unlinkability	15
7	Atomicity, No Double-Mint, No Replay	15
7.1	Atomicity	15
7.2	No Double-Mint From One Transfer	16
7.3	No Replay	16
7.4	Cross-Chain or Cross-Deployment Replay	16
8	Prover Contention and the Rollup Race	16
8.1	Who Wins When Two Minters Race?	17
8.2	Mitigation Layers	17
8.3	What Does Not Regress	17
9	Receipt Construction From the Transaction ID Alone	18
10	The Redemption (Spend), Step by Step	18
10.1	Step 0: Bob Holds a Verified Note	19
10.2	Step 1: Fetch the Commitment Tree and Build a Merkle Path	19
10.3	Step 2: Compute the Nullifier	20
10.4	Step 3: Generate the Spend Proof	20
10.5	Step 4: Submit <code>Notes.spendCoupledA2</code> On Chain	21
10.6	Step 5: BUCK ERC-20 Pays Out (the Carrying path)	23
10.7	What the Observer Sees at Spend (Mallory Returns)	24
10.8	Why Alice Cannot Steal Bob’s Note (Even Between Rooms)	24
10.9	B1 Bearer Notes	25
10.10	Recapping Property C (Only Valid Notes Redeem)	25

11 The Identity-M Spend Path (A1, A2, B1)	27
11.1 The Shared Primitives	27
11.2 A2 Identity-Targeted Spend (The Unilateral Receipt Flow)	28
11.3 A2 in Code: The Full Cycle, End to End	30
11.4 A1 in Code: The Public-Issuer Addressed Cycle	34
11.5 B1 Bearer Spend with Depositor Naming (The A2 Dual)	34
11.6 B1 in Code: The Issuer Names the Depositor	35
11.7 Summary: What the Observer Sees (Mallory)	36
12 Demurrage-Carrying Transfers	37
12.1 Two Ways to Handle Demurrage at Transfer Time	37
12.2 Why the Note Pool Needs It	37
12.3 Shipped Surface in <code>Buck.sol</code>	38
12.4 Who Else Is Carrying	38
12.5 Open Question: Identity Receipts on Carry	38
13 Putting the Guarantees Side-by-Side	39
14 Open Questions and the Identity-M Transition	39
15 Privacy Audit: Data Revealed Per Step	41
15.1 Requirements and adversaries	41
15.2 The on-chain footprint of each step	41
15.3 Why bulk-tracing and third-party privacy hold	42
15.4 Why mutual decryptability holds – the final data set	42
15.5 The colluding pair, and where each escape is closed	43
16 Appendix: The Hostile-Issuer Mint Ledger	44
17 Security Properties and the Tests That Witness Them	45
18 Cross-References	46

1 Cast of Characters

The example uses concrete names so the message flow is easy to track:

- **Alice** – the issuer. She holds 1,200 BUCK and wants to break it into six notes. She is registered with the IdentityRegistry so that her `msg.sender` address resolves to a cryptographically attested identity.
- **Bob, Carol, Dave, Eve, Frank, Grace** – six recipients. Bob will get the 1,000-BUCK note; Carol the 100-BUCK note; Dave/Eve/Frank/Grace one 25-BUCK note each. Recipients do *not* need to be on-chain known to Alice – the recipient identity binding is a field inside the commitment, not an on-chain registration step at mint.
- **The honest EVM** – every Ethereum validator running stock client software. This is the only EVM whose verdict matters. It executes the deployed `Notes.sol`, `MintGroth16Verifier.sol`, `MintVerifierAdapter.sol`, and `Buck.sol` bytecode.
- **Alice’s hostile EVM (Eve-EVM)** – a fictional entity Alice can run locally. She may patch any contract logic she wants, run any witness generator, and produce any proof bytes. The point of the design is that nothing she runs locally affects what the honest EVM accepts.
- **A passive observer (Mallory-the-watcher)** – someone scanning the public chain after the fact, trying to learn *which* note went to *whom* for *how much*. Mallory has full chain history and unlimited compute.

2 The Promise

By the end of the mint transaction Alice will have:

- A single Ethereum transaction `T_mint` on-chain whose effects are:
 - 1,200 BUCK debited from Alice and credited to the `Notes` pool address.
 - A batch of N^* opaque `uint256` commitments (here $N^* = 16$: six live plus ten dummies) folded into the note Merkle tree, moving the root from `oldRoot` to the SNARK-attested `newRoot`, and the tree size from `nextLeafIndex` to `nextLeafIndex + N^*`.
 - One `Minted(issuer=Alice, totalFace=1200e18, startIndex=K, count=N*, newRoot)` event; no per-leaf `Appended` events – the commitments themselves live in the transaction calldata.
 - One `Transfer(Alice, Notes, 1200)` ERC-20 event.
 - One Groth16 proof (256 bytes, shape-independent of N) in calldata, accepted by the on-chain per- N verifier.
- Six off-chain artifacts, one per *live* recipient, each containing the *opening* of exactly one commitment plus the mint-transaction reference. The ten dummy openings are discarded or retained privately by Alice as anonymity- set filler.

The key invariant: looking at the chain alone, Mallory sees that Alice escrowed 1,200 BUCK and folded N^* hashes into the tree. She cannot tell which hash hides which value, who any recipient is, or even whether two of the recipients are the same person – or which entries are live notes vs.

zero-value padding. Looking at his off-chain artifact, Bob sees his 1,000 BUCK is real, was minted by Alice in `T_mint`, and that the *live* siblings among the other fifteen have hidden values summing to exactly 200 BUCK (the rest are $v = 0$ padding he cannot distinguish from live notes by chain inspection alone) – but he does not learn what those values are or who holds them.

3 What’s Inside a Note (Conceptually)

A "note" is just a five-tuple known by Alice and the recipient:

```
opening_i = ( flavor_i, v_i, rho_i, idHash_i, predicate_i )
cm_i       = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

- **flavor_i** – which note variant (A1 identified cheque, A2 private cheque, B1 bearer cheque); see Notes for the menu.
- **v_i** – the face value in BUCK base units (18-decimal scaled integer).
- **rho_i** – a fresh, uniformly random nonce. This is the source of unlinkability: swapping **rho_i** changes **cm_i** to a uniformly fresh hash even if every other field is identical.
- **idHash_i** – a Poseidon-hashed handle on the recipient’s identity (e.g. a Poseidon-hash of the recipient’s registered ElGamal credential, or zero for a pure bearer note).
- **predicate_i** – a domain-separating tag for any optional spending condition (escrow, time-lock, etc.). Zero in the simplest case.

The Poseidon-5 hash gives the commitment two cryptographic properties that make the whole flow work:

- **Hiding.** Without **rho_i**, the commitment leaks no information about the other fields. Mallory sees **cm_i** and gains nothing about **v_i**, **idHash_i**, or **flavor_i**.
- **Binding.** No prover, hostile or honest, can find two distinct openings hashing to the same **cm_i** (collision-resistance of Poseidon on BN254, modelled as a random oracle in alberta-buck-proofs.org Part IV).

The on-chain contract stores only **cm_i**. Openings live exclusively off-chain.

4 The Mint, Step by Step

4.1 Step 0: Pre-flight (Off-Chain, In Alice’s Wallet)

Alice’s wallet has done all the slow work before any transaction is broadcast:

- She owns 1,200 BUCK on-chain.
- She has decided on the denomination split: [1000, 100, 25, 25, 25, 25]. $N=6$. The deployed mint pipeline pins a small set of batch sizes – recommended {16, 128, 1024} – one circom circuit and one verifier per pin. Alice’s wallet rounds her requested split up to the smallest pin $N^* \geq 6$, here $N^* = 16$, and pads with $16 - 6 = 10$ "blinded zero" openings whose **v_i** = 0. The padding leaves cost the same SNARK constraints as live notes but

contribute zero to `totalFace` and are spendable for zero BUCK; in effect they are anonymity-set filler whose openings Alice may discard. The narrative below tracks the six live notes and ignores the ten dummies (they show up in the on-chain commitments[] vector alongside the live ones, indistinguishable to Mallory).

- She has chosen an `idHash_i` for each recipient (typically a Poseidon hash of the recipient’s IdentityRegistry-bound ElGamal credential, derived once at out-of-band introduction time).
- Her CSPRNG produces a fresh `rho_i` nonce per leaf – live or padded.

She now has 16 opening tuples in memory: 6 live, 10 dummies.

4.2 Step 1: Build Commitments

For `i = 0..15` (six live, ten dummies) her wallet computes

```
cm_i = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

This is pure off-chain Poseidon arithmetic in `alberta_buck.wallet.notes`. Sixteen `uint256` field elements drop out. Their order in the eventual on-chain array is fixed by Alice’s local list – the contract preserves whatever order she submits, and the SNARK assumes leaves land in the tree at indices `[nextLeafIndex, nextLeafIndex+1, ..., nextLeafIndex+N-1]` in submission order.

Alice’s wallet must also keep a *local mirror of the rolling tree state*. The Tornado-style Merkle tree the contract maintains is parameterised by the standard `filledSubtrees[d]` for `d = 0..TREE_DEPTH-1` – the rightmost partially-built node at each level. The wallet snapshots this state from the chain (or replays it from the mint event log) immediately before assembling the proof, then derives, for each of the 16 leaves *in submission order*, the depth-20 sibling sequence the in-circuit insertion will consume. That derivation is the only delicate piece of new wallet code; everything else is unchanged from Phase-7.

4.3 Step 2: Generate the Batch-Mint Groth16 Proof

Alice’s wallet runs `scripts/snark/prove_mint_batch.js` (the Phase-7-bis successor to Phase-7’s `prove_mint.js`). The `mint_batch.circom` circuit at her chosen `pin N* = 16` takes a substantially richer witness than the Phase-7 mint did, because the Merkle-tree update has moved *into* the SNARK.

Public signals (circom emits the circuit OUTPUTS first, then the public inputs; arity `2N+4`):

```
issuerMode[i] # i in 0..N-1: per-leaf PUBLIC=1 / PRIVATE=2 (a circuit OUTPUT,
               # the deterministic flavor projection -- Notes.mint gates on it)
oldRoot       # the live note root she read from chain
newRoot       # the root after folding her N leaves
nextLeafIndex # the live tree size she read from chain
totalFace     # 1200e18 -- range-bounded to 128 bits
cm[i]         # i in 0..N-1: the commitments, exposed DIRECTLY (not a batch hash)
```

The commitments are exposed as public inputs directly (no `cmBatchHash`): each input costs ~6K gas in the EVM verifier, cheaper than recomputing a keccak over calldata on-chain, and far cheaper than an in-circuit Poseidon over `N` elements. (A2 / private-issuer batches use a *separate* circuit, `mint_batch_a2`, that exposes each leaf’s `eIss` instead of `issuerMode`; see *The Redemption* and the Decryptability companion.)

Private witness (per-leaf and per-level):

```

For i in 0..N-1:
  flavor[i], v[i], rho[i], idHash[i], predicate[i]    # the opening
  pathSiblings[i][d] for d in 0..TREE_DEPTH-1        # 20 siblings per leaf

```

The `pathSiblings` for leaf `i` are derived by the wallet from its local `filledSubtrees` mirror plus the previously-folded leaves of *this same batch* (leaves `0..i-1` sit at known positions and contribute their Poseidon hashes to `i`'s path on the right). The wallet does this work; the chain consumes only `oldRoot`, `newRoot`, and the calldata `cms[]`.

In-circuit constraints:

1. **Per-leaf opening.** For each `i`, `cm[i] === Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])`. `N` Poseidon-5 evaluations, `N` equality constraints.
2. **Range bounds.** Each `v[i]` and `totalFace` passes through `Num2Bits(128)`.
3. **Sum conservation.** `sum(v[0..N-1]) === totalFace`.
4. **Rolling Merkle insertion.** The circuit threads `oldRoot` and `nextLeafIndex` through `N` successive single-leaf insertions, mirroring the on-chain Phase-7 `Notes._insert` logic but in-circuit: at each step it folds `cm[i]` up the tree using `pathSiblings[i][*]` and `Switcher(...)` per level, advances the rolling state, and finally asserts the resulting root equals `newRoot`. `N x TREE_DEPTH` Poseidon-2 hashes ($\sim 4.3K$ constraints per leaf).
5. **Flavor range + issuer mode.** Each `flavor[i]`, $\in \{A1=1, A2=2, B1=3\}$, and `issuerMode[i]` is the deterministic projection (`A2` $\rightarrow$ PRIVATE, else PUBLIC), exposed as a public output. This is what makes a bearer leaf provably public-issuer (and a private-issuer bearer note unmintable – see the Decryptability companion, *Why B2 is impossible*).

Constraint counts work out to roughly **21,800 R1CS per leaf** (opening + range-bound + dual Merkle walk: one path consumed to attest `oldRoot`, one to advance `newRoot`); the shipped pinned-`N` set measures 21,914 R1CS at `N=1`, 348,705 at `N=16`, and 697,281 at `N=32`. The doubling per pin holds linearly through the projected sizes (`N=128` $\sim 2.8M$ R1CS, `N=1024` $\sim 22M$). At `N=16` the 2^{19} PTAU and ~ 6 s prover wall on a dev laptop are comfortable; at `N=128` it is rapid snark-and-minutes; at `N=1024` it is hours plus a multi-GB zkey. The `scripts/snark/setup.sh` ceremony emits one verifier per pin.

The witness is processed by snarkjs's Groth16 prover. Tens of seconds to minutes later the wallet has a fixed-size proof object (`pi_A: G1`, `pi_B: G2`, `pi_C: G1`), encoded as ABI-static (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) – exactly 256 bytes regardless of `N`.

Observation. The Groth16 proof reveals *nothing* about the witness. Soundness guarantees that no `proofBytes` can satisfy the verification equation unless a witness exists; zero-knowledge guarantees that the verification equation reveals nothing about which witness was used. Mallory, even with the proof in hand, cannot back out `v[i]`, `rho[i]`, `idHash[i]`, `flavor[i]`, `predicate[i]`, or the per-leaf Merkle paths – only `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, the `cm[i]`, and the per-leaf `issuerMode[i]` (flavor-class: public vs private, no identity) leak.

4.4 Step 3: Submit Notes.mint On Chain

Alice now broadcasts a single transaction `T_mint` calling

```

// Minting is gated-only: a nameable issuer Identity is bound to every batch.
// Alice (a public Corporate Identity here) uses the PUBLIC overload:
notes.mint(

```

```

    proofBytes,
    oldRoot, newRoot, nextLeafIndex,
    1200e18, // totalFace
    [cm_0, cm_1, ..., cm_15], // cms.length == N* (here 16)
    [PUBLIC, ..., PUBLIC], // issuerMode (a SNARK public input of mint_batch)
    issuerSig // Schnorr by Alice's registered key over keccak(cms)
);
// A private (A2) issuer uses the other overload instead:
// notes.mint(..., cms, issuerMode, a2Bindings) // routes the proof to mint_batch_a2
// // and ties each leaf's eIss
// There is no unbound mint path -- an unnameable note cannot be created.

```

The honest EVM walks `Notes.mint` (the `PUBLIC` overload):

```

// Stale-state guards + field bounds (_verifyMintOrRevert).
require(cms.length > 0, "Notes: empty mint");
require(issuerMode.length == cms.length, "Notes: issuerMode/cms length");
require(oldRoot == roots[currentRootIndex], "Notes: stale oldRoot");
require(nextLeafIndex == self.nextLeafIndex, "Notes: stale nextLeafIndex");
require(newRoot != 0 && newRoot < FIELD_R, "Notes: bad newRoot");

// SNARK verify: the proof binds issuerMode + cm[] + the roots. The
// MintVerifierAdapter dispatches by cms.length to the per-N Groth16 verifier;
// public-signal vector is [issuerMode[0..N), oldRoot, newRoot, nextLeafIndex,
// totalFace, cm[0..N)] (outputs lead), arity 2N+4.
require(mintVerifier.verifyMint(
    proofBytes, issuerMode, oldRoot, newRoot, nextLeafIndex, totalFace, cms
), "Notes: bad mint proof");

// Public-issuer binding (gated-only): every leaf must be PUBLIC-mode,
// msg.sender must be a registered PUBLIC Identity, and issuerSig must be a valid
// Schnorr over keccak256(cms). (The PRIVATE-mode overload instead routes the
// proof to mint_batch_a2 and verifies one A2 re-encryption binding per leaf,
// tying each committed leaf's eIss.)
require(identityRegistry.isPublicIdentity(msg.sender), "Notes: public-mode leaf needs public issuer");
require(identityRegistry.verifyIssuerSchnorr(
    msg.sender, keccak256(abi.encodePacked(cms)), issuerSig,
), "Notes: bad issuer binding");

// Pull BUCK only after the proof is accepted, then advance (_advanceAndPull).
require(buck.transferFrom(msg.sender, address(this), totalFace), "Notes: transfer failed");

currentRootIndex = (currentRootIndex + 1) % ROOT_HISTORY_SIZE;
roots[currentRootIndex] = newRoot;
self.nextLeafIndex += cms.length;
noteFaceSum += totalFace;

// Single batch event; cms[] live in calldata.
emit Minted(msg.sender, totalFace, /*startIndex=*/ nextLeafIndex,
    /*count=*/ cms.length, newRoot);

```

Five things happen in strict order, and **all of them are atomic** (one transaction, one EVM state transition):

- a. **Stale-state guards.** The contract asserts that `oldRoot` equals the current ring-buffer head and `nextLeafIndex` equals the live tree size. Either fails if another mint landed since Alice's wallet snapshotted state (more on this under **Prover Contention and the Rollup Race** below).
- b. **Calldata binding.** The `cms[]` are exposed **directly** as Groth16 public inputs (no `cmBatchHash` digest), alongside the per-leaf `issuerMode[]` the circuit emits, so any tampering with `cms[]` or `issuerMode[]` en route from the prover to the chain mismatches the SNARK and the verify call rejects.
- c. **SNARK verify.** The `MintVerifierAdapter` dispatches by `cms.length` to the per-N verifier (one circuit per pin, one verifier contract per pin), unpacks the 256-byte `proofBytes` into Groth16 form, builds the $2N+4$ -entry public-signal vector `[issuerMode[0..N)`,

oldRoot, newRoot, nextLeafIndex, totalFace, cm[0..N]) (circuit outputs lead), and invokes the appropriate `MintBatchN${N}Groth16Verifier`. If the verifier returns false, the entire transaction reverts – nothing is written, no BUCK moves. d. **BUCK escrow.** `buck.transferFrom` debits `totalFace` from Alice and credits the Notes pool. Allowance-driven pull, gated by Alice’s prior `approve(notes, 1200e18, E_notes, pi_CP)`. This CP-bound `approve` is mandatory: Alice is a private EOA, and mutual decryptability requires a per-pair CP fragment for every private party (the `identityHash` fallback is only valid for Public-Identity contracts). The pool was bound under a Public Identity (`bindContract(notes, pk, E, true)`), so the pool side falls back to `_identityHash(notes)` without requiring a fragment from the pool. The Notes operator decrypts Alice’s CP ciphertext from the `IdentityExchange` event under subpoena. e. **Root advancement.** The contract writes `newRoot` into the next ring-buffer slot, advances `currentRootIndex` and `nextLeafIndex`, accumulates `noteFaceSum`, and emits a single `Minted(issuer, totalFace, startIndex, count, newRoot)` event. The new root immediately becomes the live root that the next minter must read.

If any of (a)-(e) fails – stale root, stale leafIndex, bad proof, insufficient BUCK, missing approval – the entire transaction reverts and the chain state is indistinguishable from no submission at all. **Alice gets one shot per transaction;** she cannot retry a partial state.

Note on duplicate commitments. Phase-7 carried an explicit `commitmentExists[cm]` mapping that reverted on duplicate insertion. The batch-mint contract drops it – not because duplicates are harmless, but because they are now *economically self-punishing*: if Alice submits the same opening twice (whether across two batches or twice in one), she pays `totalFace` on each occurrence, the leaves land at distinct tree indices, but the spend nullifier `Poseidon3(rho, idHash, 4242)` is *deterministic in the opening*. After the first spend nullifies `cm`, the second copy is unspendable forever. Net result: Alice burns BUCK she could have kept. The chain’s job is value conservation, not saving Alice from a buggy `rho` RNG. See **Why Outside Observers Cannot Link** below and the per-attack-vector audit in alberta-buck-notes-rollup-mint.org.

4.5 Step 4: Off-Chain Note Delivery

After `T_mint` confirms, Alice transmits each live opening to its intended recipient by any out-of-band channel: an encrypted message, a printed QR code, a USB stick. The payload to recipient `i` is:

```
note_artifact_i = {
  txHash:      T_mint,                # Ethereum transaction ID
  leafIndex:   startIndex + i,        # absolute position in the commitment vector
  cm:          cm_i,                  # for cross-checking against calldata
  opening:     ( flavor_i, v_i, rho_i, idHash_i, predicate_i ),
  meta:        { issuer: Alice's IdentityRegistry handle, batchSize: N*, ... },
}
```

The chain itself never sees `opening`. `cm_i` is already folded into `newRoot`; the artifact carries it explicitly only as a convenience so the recipient does not have to recompute it before locating it in the batch’s calldata `cms[]`.

The ten dummy openings Alice generated to pad to the pinned `N=16` circuit receive no artifact; they are discarded (or, at Alice’s option, retained privately as unspendable anonymity-set filler). From the recipient’s perspective the presence of dummies is invisible: Bob’s artifact only names his own leaf, and the batch total `totalFace = 1,200 BUCK` is exactly the sum of the live `v_i`.

4.6 Step 5: Recipient Verification (Offline, Independent)

Bob receives his artifact for the 1,000-BUCK note. Without trusting Alice and without any help from an indexer, he runs:

1. **Recompute and check.** Bob computes `cm_check = Poseidon5(flavor, 1000e18, rho, idHash, predicate)` and asserts `cm_check == cm`. If this fails, Alice lied about the opening; Bob discards the artifact and demands a real one. (Binding of Poseidon makes substitution impossible.)
2. **Look up the mint transaction.** Bob reads `T_mint` on-chain, parses the call data into `(proofBytes, oldRoot, newRoot, nextLeafIndex, totalFace, cms[])` and reads `msg.sender` from the transaction envelope.
3. **Confirm msg.sender is Alice.** Cross-check `msg.sender` against the IdentityRegistry: it must resolve to Alice's PS-credentialed identity. If Bob wanted an A1 identified cheque, the issuer field of his opening must match.
4. **Confirm calldata membership.** Assert `cms[leafIndex - nextLeafIndex] == cm` (where the artifact's `leafIndex` is the absolute position). The chain's acceptance of `T_mint` confirms that every entry in `cms[]` was folded into `newRoot` and that `newRoot` was added to the root ring buffer. Bob therefore has chain-anchored evidence that `cm` is officially a commitment in the pool.
5. **Confirm batch totals.** Read `totalFace` from the call data. The chain's acceptance of `T_mint` is itself a proof that the Groth16 verifier accepted – which means `sum(v[0..N-1]) == totalFace` holds in the witness. Bob does not need to re-verify the proof; the EVM already did. But if he wants belt-and-suspenders, he can re-run `MintGroth16Verifier.verify(proofBytes, [oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash])` locally against the same on-chain bytecode – and `cmBatchHash` he recomputes from the calldata `cms[]` using the same `keccak256(abi.encodePacked(cms))` the contract uses.

After (1)-(5) Bob knows:

- His note encodes exactly 1,000 BUCK. (Step 1.)
- It was minted in `T_mint` by Alice. (Steps 2, 3.)
- It is one of N^* (here 16) commitments folded into the tree by that mint, the others being `cms[] \ cm`. (Step 4.)
- The *live* values among the other siblings sum to exactly $1200 - 1000 = 200$ BUCK (the residual of `totalFace` once Bob subtracts his own `v`); the batch may also contain dummy openings with `v = 0`. (Step 5 + the per-note opening relation: he knows *his* value, the chain attests to the *sum*.)
- He learns nothing else about the other notes – not their individual values, not their recipients, not their flavors, not which are live and which are padding. Each `cm_j` is hiding under a fresh `rho_j` he does not have.

This is the unlinkability promise made concrete: each recipient can pin down their own slice and the batch total, without learning anything about siblings.

5 Why Alice Cannot Cheat (Even With a Hostile EVM and Prover)

Alice runs whatever software she wants on her laptop. She can fork the snarkjs witness generator, patch the circom compiler, run Anvil with a malicious `MintGroth16Verifier` that always accepts. None of it matters, because the truth on the public chain is determined by what *the honest validators* run. Three pillars prop up the soundness story:

Note (shipped detail). The pillars below were written when the mint proof sealed a single `cmBatchHash = H(cm_0..cm_{N-1})`. The shipped circuit exposes the `cm[i]` **directly** as public inputs (and adds a per-leaf `issuerMode[i]` output) – so wherever a pillar says `cmBatchHash`, read "the `cm[]` (and `issuerMode[]`) the proof seals." The argument is unchanged: the public inputs, whatever their exact list, are bound into the proof and re-checked by the on-chain verifier. The public-signal vector is `[issuerMode[0..N], oldRoot, newRoot, nextLeafIndex, totalFace, cm[0..N]]`.

5.1 Pillar 1: The On-Chain Verifier Is Fixed Code

`MintGroth16Verifier.sol` (one per pinned `N`) was emitted from the Groth16 verification key produced by the trusted setup (Phase 9 for production; dev-only entropy in `scripts/snark/setup.sh` today). Its bytecode is deployed once and immutable. Validators run whatever bytecode sits at that address; Alice cannot tamper with it without consensus-breaking the chain.

Knowledge soundness of Groth16 means: for every `proofBytes` the verifier accepts on public inputs `[oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash]`, there exists a witness `(flavor[i], v[i], rho[i], idHash[i], predicate[i], pathSiblings[i][*])_i` such that:

- `cm[i] = Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])` for all `i`
- each `v[i]` and `totalFace` fit in 128 bits and `sum(v[i]) = totalFace`
- folding `cm[0..N-1]` into the tree rooted at `oldRoot`, starting at position `nextLeafIndex`, using `pathSiblings[i][*]` as the right-of-insert context, yields exactly `newRoot`
- `cmBatchHash = H(cm[0], ..., cm[N-1])`.

Alice is free to *find* such a witness any way she likes – she’s the prover. But she cannot find a witness for a *false* statement; that’s what soundness prohibits.

5.2 Pillar 2: BUCK Escrow Is Inside the Same Atomic Call

The proof check, the BUCK pull, and the root advancement all live in the same `Notes.mint` function. An EVM transaction is all-or-nothing: the only two outcomes are "the entire call’s state changes commit" or "the entire call’s state changes revert".

This means Alice cannot, for instance:

- Verify a proof, then refuse to pay – the verifier acceptance and the `transferFrom` live in the same call, gated by the same `require`.
- Pay 1,200 BUCK but then get away with advancing only part of the tree – `newRoot` is SNARK-attested as the deterministic fold of `all` of `cms[]` onto `oldRoot` at `nextLeafIndex`, and the contract writes it in one store.

- Replay a prior `proofBytes` – the SNARK is bound to (`oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cmBatchHash`) and the contract’s stale-state guards reject any submission whose `oldRoot` no longer matches the live root or whose `nextLeafIndex` no longer matches the live tree size.
- Submit the same opening twice *in one batch* to double-spend at spend time – the nullifier is deterministic in (`rho`, `idHash`), so whichever duplicate lands first at spend will consume both. (See the duplicate-commitment note under Step 3.)

5.3 Pillar 3: Public Inputs Are Sealed Into the Proof

The Groth16 verifier consumes the public-signal vector [`oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cmBatchHash`]. The proof was generated against the specific values Alice committed to before submission. If Alice tries to submit the proof with a different `totalFace`, a different `newRoot`, a different `nextLeafIndex`, or a permuted `cms[]` whose keccak no longer matches `cmBatchHash`, the verifier rejects.

Note that `cms[]` itself is **not** a public input to the SNARK – the proof binds only to its hash `cmBatchHash`. The contract is the component that recomputes that hash from calldata on every call, so any mismatch between the proved hash and the recomputed hash reverts at verify time.

The refreshed `test/MintVerifier.t.sol` should cover at least these tamper paths, each expected to revert with `Notes: bad mint proof` or `Notes: stale oldRoot` or `Notes: stale leafIndex` or an ABI decode error:

- `totalFace + 1`
- `newRoot` flipped by one bit
- `oldRoot` spoofed to a stale value
- `nextLeafIndex` spoofed to a fictional size
- `cms[0] ^ 1` (commitment tampered in calldata after proving)
- `cms[]` length differs from the dispatched verifier’s pinned `N`
- malformed 256-byte `proofBytes`

Soundness is not a proof of *absence* of attack, but every documented attack surface is closed by a test that triggers a verifier reject.

5.4 Putting the Pillars Together: The Adversarial Walk

Suppose hostile-Alice tries to mint two **contradictory** batches that both claim to be the canonical breakdown of her `T_mint` transfer. She would need:

- Two different `proofBytes` and/or `cms[]`, both passing the on-chain verifier.

That requires either:

- Two valid Groth16 witnesses for the same public inputs (allowed – but they yield the same `cms`, so they are not *contradictory*), or

- Two valid Groth16 witnesses with *different* public inputs. Allowed too, but each one demands its own `transferFrom` for its own `totalFace` – **that** is the duplicate-spend problem ERC-20 solves. Alice cannot transfer the same 1,200 BUCK twice without holding 2,400 BUCK to start with.

So a single 1,200-BUCK escrow yields exactly one accepted batch. The only freedom Alice has is *which* batch she lays down – which denominations and which recipients – and that freedom is hers to keep, by design.

What about hostile-Alice trying to mint *more* commitment value than she escrowed? She would need a witness whose `sum(v[i])` equals the integer `totalFace` *that the EVM sees*, but whose individual `v[i]` values *actually* sum to more. In `F_r` with `r ~ 2^254` and no range check on `v[i]`, the constraint `sum(v[i]) == totalFace` would be satisfiable by witnesses whose integer sum differs from `totalFace` by a multiple of `r`. `mint_batch.circom` closes this with a `Num2Bits(128)` range constraint on each `v[i]` and on `totalFace`, mirrored by `spend.circom`'s check on `face` and the witness `v`. A 128-bit cap is comfortably above any plausible BUCK denomination and well below `r/2`, so an integer overshoot can never wrap.

What about hostile-Alice trying to mint a commitment that is *already* in the tree (to collide on a nullifier she does not control, or to attempt a hash-collision attack)? She cannot collide on a nullifier without knowing somebody else's opening – the nullifier is a Poseidon-PRF output, so forging a collision is as hard as inverting Poseidon. She *can* re-mint her own opening (the contract no longer blocks duplicates), but doing so forces her to pay `totalFace` a second time and buys her nothing: the nullifier is deterministic in the opening, so whichever of her two copies she spends first permanently nullifies both. This is the value-conservation property in action – the chain does not need a `commitmentExists` check because the nullifier mechanism already collapses duplicate openings to one spend, and Alice bears the cost of any duplication.

Full per-attack-vector analysis (twelve attack vectors A1-A12) is in the pivot design memo alberta-buck-notes-rollup-mint.org. Every hostile-Alice guarantee from the Phase-7 per-leaf insertion design survives under the batch-mint pivot; the only behavioural change is that duplicate-commitment rejection moves from an explicit on-chain check to economic self-enforcement via the deterministic nullifier.

6 Why Outside Observers Cannot Link

Mallory has full read access to the chain after `T_mint` confirms. What does she see?

```
T_mint:
  from: Alice (a registered IdentityRegistry account)
  to:    Notes (a system-public contract)
  value: 0 (BUCK is ERC-20, not ETH)
  data: Notes.mint(proofBytes, oldRoot, newRoot, nextLeafIndex,
                  1200e18, [cm_0..cm_15])

events emitted (in order):
  Buck.Transfer(Alice, Notes, 1200e18)
  Notes.Minted(Alice, 1200e18, startIndex=K, count=16, newRoot)
```

The batch-mint contract emits one `Minted` event per batch rather than per-leaf `Appended` events; the commitments themselves live in calldata for off-chain provers and wallets to read. (Calldata is about an order of magnitude cheaper per byte than LOG data, which is part of why the N-independent cost bound holds.)

What can Mallory infer?

- That Alice (publicly) escrowed 1,200 BUCK. **Yes** – her address is `msg.sender`.
- That sixteen new commitments entered the pool at indices `[K..K+15]`. **Yes** – Minted announces `startIndex` and `count`, and `cms[]` is in `calldata`.
- Which commitment is worth *how much*. **No** – each is a fresh Poseidon5 output hidden by an independent `rho_i`. Without a `rho_i`, distinguishing the 1,000-BUCK hash from the 25-BUCK hash (or from a 0-BUCK dummy) is computationally equivalent to inverting Poseidon on a uniform field element, modelled here as a random oracle.
- /Which entries are live notes and which are dummies.* **No** – the padding leaves are constructed with the same Poseidon-5 shape and a fresh `rho`; nothing about them is chain-distinguishable from a live `cm` with a small `v`.
- Who holds each live note. **No** – `idHash_i` is also inside the hash.
- Whether two commitments share a recipient. **No** – two notes addressed to the same `idHash` but with distinct `rho_i` produce uniformly independent `cm_i`, so the chain shows nothing distinguishable from independent draws.

What about *linking back to a specific note artifact* that Bob later possesses? Mallory sees `cm_0..cm_5` published by Alice in `T_mint`. Bob's artifact references some `cm_i` and `T_mint`, but the artifact lives off-chain. Mallory cannot get the artifact unless Bob (or Alice) shows it to her, and the chain alone gives her no way to single out `cm_i` as "the one Bob holds" against the other five.

The remaining linkability surface is the issuer. `msg.sender` is publicly Alice, so every commitment in the batch carries a public **issuer** tag. This is intentional and desired: a recipient must be able to verify who minted his note. The privacy goal is unlinkability of recipients and amounts, not of issuers.

6.1 Spend-Time Unlinkability

The shipped spend circuit (Phase 7) lets Bob deposit his note without revealing *which* `cm` in the on-chain set he opened. The spend SNARK proves "I know an opening to *some* commitment in the Merkle tree under the public `noteRoot` that hashes to this nullifier", with `cm` as a private witness rather than a public input. Unlinkability of *spend events* to *mint events* is now a chain-observable property: Mallory sees a fresh nullifier, a public face, and a recipient – nothing that points back to a specific commitment in the tree. The full walkthrough of the redemption flow is in **The Redemption (Spend), Step by Step** below.

7 Atomicity, No Double-Mint, No Replay

Three properties together rule out duplicated or partial mints:

7.1 Atomicity

Already covered above: `Notes.mint` performs proof verification, BUCK pull, and commitment appends in one EVM call. Either all happen or none.

7.2 No Double-Mint From One Transfer

The transfer and the commitments are bound in the same call: there is no "pre-deposit, then mint" or "mint, then deposit later" flow that an attacker could desynchronize. Each `Notes.mint` invocation does its own `transferFrom`; each `transferFrom` debits Alice's BUCK balance once. Two minting transactions require two `transferFrom` debits, which require Alice to actually hold $2 * \text{totalFace}$ BUCK.

7.3 No Replay

A literal replay – broadcasting the exact same `T_mint` calldata in a new transaction – fails for three independent reasons:

- a. **Stale-state guards.** The contract checks `oldRoot == roots[currentRootIndex]` and `nextLeafIndex == self.nextLeafIndex`. The first mint has already advanced both, so the replay's `oldRoot` and `nextLeafIndex` public inputs no longer match the live chain state. The contract reverts before the SNARK is even called, with `Notes: stale oldRoot` or `Notes: stale leafIndex`.
- b. **SNARK binding.** Even if a future minter coincidentally re-created the same `oldRoot` and `nextLeafIndex` state (which is prohibited by the monotonic advancement of `nextLeafIndex`), the SNARK was bound to the specific (`cmBatchHash`, `newRoot`) pair of the prior batch. A replay would fold the same `cms[]` a second time and produce a different `newRoot` than the SNARK attested to, breaking the rolling-tree relation.
- c. **Allowance and balance.** Even setting the above aside, the second mint would re-debit Alice's BUCK and re-consume her allowance to the `Notes` pool – economic-level double-escrow protection on top of the structural guards.

Replay of the same opening across batches (the replacement of the Phase-7 `commitmentExists` check) is structurally allowed but economically self-punishing: Alice escrows `totalFace` again, the duplicate commitment lands at a fresh leaf index, and the deterministic nullifier `Poseidon3(rho, idHash, 4242)` means the first spend nullifies every copy. Alice pays twice for a spend-once outcome.

7.4 Cross-Chain or Cross-Deployment Replay

The `MintGroth16Verifier` verification key is fixed by the trusted-setup ceremony for that deployment. A proof generated against the dev-key in `scripts/snark/setup.sh` will not verify against a production-key contract on mainnet, and vice versa. Public inputs are also sealed into the proof: a proof valid on Sepolia for `totalFace=1200` cannot be reused on mainnet because the on-chain verifier address (and key) differ.

The only cross-context vulnerability would be a *chain-id collision* on identical verification keys, which the production setup ceremony precludes by construction.

8 Prover Contention and the Rollup Race

The batch-mint pivot introduces one genuinely new operational concern that did not exist in the Phase-7 per-leaf insertion design: a mint transaction is now a *rollup-style* transaction. Alice's proof is bound to the specific `oldRoot` and `nextLeafIndex` she snapshotted when she began proving; if any other mint lands between her snapshot and her submission, those values go stale and her transaction reverts at one of the stale-state guards.

This is not a *safety* concern – Alice loses no BUCK (the contract reverts before `transferFrom` runs) and produces no bad on-chain state. It is a *liveness and efficiency* concern: Alice wasted the prover time spent constructing a proof that is no longer accepted, and has to re-prove against the fresh state.

8.1 Who Wins When Two Minters Race?

When Alice and Bob both submit `T_mint_A` and `T_mint_B` targeting the same `oldRoot`:

- One of the two (by block ordering) lands first and updates the root.
- The other transaction executes, discovers the state-state mismatch, and reverts at the `require(oldRoot == roots[currentRootIndex])` line before any BUCK moves.

Transaction fees are paid by the loser for the reverted transaction (gas for the revert path, not for the full mint). The loser then re-proves against the new `oldRoot` and `nextLeafIndex` and resubmits. The new proof folds *their* commitments onto the root that *now* includes the winner's commitments – no commitment has been lost; the loser's tree-position is just different from the one they originally predicted.

8.2 Mitigation Layers

Three mitigation layers, in ascending complexity:

1. **Optimistic retry (wallet-level).** On revert, the wallet re-fetches (`currentRoot`, `nextLeafIndex`) from the chain, rederives the sibling paths for the new insertion positions, re-runs the prover, and resubmits. For small N this is fast enough to not matter. This is the recommended initial deployment strategy.
2. **Retry budget and back-off (wallet-level).** For large N where prover time is nontrivial (minutes to hours at $N=1024$), the wallet should expose a "chance of contention" estimate to the user and offer to batch at the smallest pinned N that fits the job, or to enqueue the mint on a private relay channel during low-traffic windows.
3. **Sequencer / proof aggregation (future).** If real-world contention warrants it, an external sequencer can batch multiple users' mints into a single proof per slot, preserving the N -independent on-chain cost while amortising prover time. Sequencer capture is a separate trust axis and is deferred; the shipped protocol is sequencer-free.

8.3 What Does Not Regress

- **Atomicity.** A reverted mint leaves *no* chain-visible state change; it is strictly indistinguishable from no submission.
- **Value conservation.** No reverted transaction debits BUCK, so there is no way for a loser to pay the pool without appending commitments.
- **Unlinkability.** A re-submitted mint on a fresh root still produces the same (anonymity-preserving) calldata and event shape; Mallory cannot tell a retry from a first attempt.
- **Spend-time invariants.** The spend circuit takes the current `noteRoot` (or any recent-window root); a late-arriving mint does not invalidate any previously-proved spend.

The rollup race is the price of replacing $\sim 800K$ gas/leaf with ~ 500 gas/leaf, and for practical block-rate contention it is negligible.

9 Receipt Construction From the Transaction ID Alone

A practical question: months later, Bob wants to prove to Carol that his note was legitimately issued. He needs no help from Alice and no live indexer:

1. Bob fetches `T_mint` from any Ethereum archive node.
2. He decodes the call data into (`proofBytes`, `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cms`) using the standard `Notes.mint` ABI.
3. He notes `msg.sender` and resolves it to Alice's identity via the IdentityRegistry on the same chain (a single `eth_call`).
4. He locates his `cm` in `cms[]` at `leafIndex - nextLeafIndex` and asserts equality.
5. He recomputes `Poseidon5(flavor, v, rho, idHash, predicate) == cm` from the opening.
6. (Optional.) He recomputes `cmBatchHash = keccak256(abi.encodePacked(cms))` and re-runs `MintGroth16Verifier.verify(proofBytes, [oldRoot, newRoot, nextLeafIndex, totalFace, cmBatchHash])` against the per-N verifier keyed by `cms.length`, convincing himself the EVM did its job.

The "receipt" Carol receives is therefore (`T_mint`, `opening`, `idHash` mapping). Its validity is independently checkable against the public chain. Nothing here requires ongoing cooperation from Alice – which is exactly the property a receipt should have.

10 The Redemption (Spend), Step by Step

This section walks the *inverse* of the mint. Bob holds the opening for the 1,000-BUCK note `cm_0`; he wants to redeem it for 1,000 BUCK credited to his on-chain account, without revealing *which* commitment in the pool he consumed.

There is **one** spend path, the *Identity-M-bound deposit*: Bob redeems by proving he is the KYC-registered Identity the note names – *not* a particular account – so he may use *any* Fountain account bound to his Identity point M_{rec} , and he reveals no Identity to the chain. Every spend composes the same two halves over a single committed point P :

1. **The note proof** (this section, Steps 1–3): the kept, flavor-agnostic `spend.circom` Groth16 proves the note's commitment `cm` is in the pool tree under a public `noteRoot` and emits a fresh, well-formed `nullifier` – hiding *which* commitment was consumed. `Notes.spendCoupled{A1,A2,B1}` all reuse `spendVerifier.verifySpend` for exactly this.
2. **The identity binding** (Step 4): an EIP-196 Okamoto sigma + a membership proof bound to the *same* P , proving the spender is a registered Identity. The flavor selects which sigma and what P commits – the issuer's M_I (A2), or the spender's own $M_{\text{rec}}/M_{\text{dep}}$ (A1/B1). This is detailed per-flavor in 11; the walk below uses Bob's **A2** note (addressed, private issuer) as the running example.

The earlier account-pinned A-spend (the Phase-8 `spend_a.circom` + `Notes.spendACP` + `verifySpendCP` CP-DLEQ, and the unbound generic `Notes.spend`) has been **retired**: every note is A1, A2, or B1, and every spend goes through a coupled, membership-bound deposit. Only the flavor-agnostic `spend.circom` note proof survives, now shared by all three.

Three properties of the flow are worth stating up front, because they shape every step:

- **The anonymity set is the full pool, not only same-denomination.** The spend proof takes `cm` as a *private* witness under a public `noteRoot` – the chain-observable anonymity set is every unspent commitment in the tree. The **face value is a public output**, because the BUCK contract must know how much to pay out; amount-level linkability narrows the effective set to commitments whose hidden `v` could equal the revealed **face**. Same-denomination issuance is therefore the strongest-privacy regime, but the underlying cryptographic hiding is over the whole tree, not over a denomination class.
- **The BUCK ERC-20 does not verify the SNARK itself.** The authorization chain is: valid SNARK + accepted root + fresh nullifier (+ for A-spends, valid identity binding) → the relevant `spendCoupled{A1,A2,B1}` entrypoint proceeds → the entrypoint invokes `buck.transfer(recipient, face)` from the pool (Buck routes it through the Carrying path). The proof is a capability token for entry into the appropriate spend method; the resulting ERC-20 movement is a routine pool-to-recipient call gated by that entry. Since the `Notes` pool is a system-public account in the IdentityRegistry, the pool’s outbound transfer does not itself require a Chaum-Pedersen receipt – the recipient’s own registration is what makes them acceptable as a transfer destination.
- **Public inputs are bound into the proof, including `chainId`.** `spend.circom` ghost-binds `recipient` and `chainId` (via `x*x` rows) so Groth16’s `IC[]` commitment makes them non-malleable from the mempool. A front-runner who steals Bob’s `proofBytes` and resubmits with a different `recipient` or on a different `chainId` is rejected by the verifier.

10.1 Step 0: Bob Holds a Verified Note

From the mint walk (**Step 5: Recipient Verification**) Bob already has:

```
note_artifact_0 = {
  txHash:    T_mint,
  index:     K + 0,
  cm:        cm_0,
  opening:   ( flavor_A1, 1000e18, rho_0, idHash_bob, predicate_0 ),
  meta:      { issuer: Alice, ... },
}
```

He has independently confirmed that `cm_0` is in `Notes.commitments[]` at index `K` and that `Poseidon5(opening)` reproduces it. The opening has been sitting in his wallet for weeks, months, or years – addressed A-notes are cryptographically safe for indefinite holding (`Notes`).

10.2 Step 1: Fetch the Commitment Tree and Build a Merkle Path

`Notes.sol` maintains a Tornado-style incremental Poseidon-T3 Merkle tree of `TREE_DEPTH = 20` (max ~1M leaves). Under Phase-7-bis the per-leaf insertion math lives inside the mint SNARK; the contract just records the SNARK-attested `newRoot` at each `mint()` call. The current root is exposed as `noteRoot()`. Two things happen:

- a. Bob (or, more practically, any light-node helper running off-chain) reconstructs the tree off-chain. The commitments themselves are available from each mint transaction’s calldata `cms[]`, keyed by the `Minted(issuer, totalFace, startIndex, count, newRoot)` event that indexes the batch. A replayer walks all `Minted` events in order, takes the `cms[]` slice out of each mint’s calldata, and appends them to its local tree using the same `ZERO_VALUE` and Poseidon-T3 wrapper the in-circuit insertion uses. Bob’s Merkle path for `cm_0` – a sequence of sibling hashes plus left/right indices up to the root – falls out of that rebuilt tree. The path is purely public information;

everyone can reproduce it, and Bob does not have to trust any indexer because calldata is canonical. b. Bob reads the current `noteRoot()` via `eth_call`. He may instead pin any root in the recent-roots window: `Notes.sol` retains the last `ROOT_HISTORY_SIZE = 30` roots in a ring buffer (`roots[]`, advanced once per inserted leaf), and the on-chain `isAcceptedRoot()` scans that ring backward from `currentRootIndex`. This is the standard zk-rollup hygiene (Zcash, Tornado Cash) for tolerating prover latency: a spend assembled against a slightly-stale root remains acceptable so long as fewer than 30 mints have landed in the meantime.

10.3 Step 2: Compute the Nullifier

The shipped `spend.circom` derives the nullifier as a Poseidon-3 hash of the witness `rho`, the witness `idHash` bound at mint, and a constant tag:

$$nf = \text{Poseidon}_3(\rho_0, \text{idHash}_0, 4242).$$

The tag (4242) is a fixed nonzero field constant that domain-separates nullifier preimages from the Poseidon-5 commitment preimages, so a structural collision between a nullifier and a commitment is impossible. Three properties follow:

- **Deterministic.** The same `(rho, idHash)` always yields the same `nf`. So Bob can only spend `cm_0` once; a second attempt re-derives the same `nf` and is rejected by `nullifiers[nf] = true` on the first spend.
- **Unforgeable without the opening.** An attacker who sees `cm_0` and the tree cannot compute `nf` without `rho_0` and `idHash_0`, both of which live inside the opening (and `rho_0` is not derivable from the recipient's identity alone).
- **The note proof binds the opening and nullifier, not the identity.** The flavor-agnostic `spend.circom` treats `idHash` as a witness scalar – it does not constrain it to a registered Identity. That binding is the *separate* Step-4 deposit gate: the EIP-196 deposit-coupling (A1/A2) or depositor-binding (B1) sigma proves the spender is bound to the named Identity scalar, and the G1-tie membership – bound to the sigma's committed point P – proves that Identity is registered. Splitting the identity binding out of the SNARK (into cheap EIP-196 sigmas, ~36K gas) avoids ~+250K R1CS of native BN254 $\mathbb{G}_1$ arithmetic in non-native circom. And because the deposit is *identity*-keyed (not account-keyed), an addressed note survives the loss of any single account key: re-register a fresh Fountain account on the same M_{rec} and deposit (recovery, not finality).

The nullifier is the chain-observable anchor of the spend. Publishing it marks `cm_0` as consumed from the tree's *semantic* point of view, even though the tree itself remains append-only and nothing is deleted.

10.4 Step 3: Generate the Spend Proof

Bob's wallet runs `scripts/snark/prove_spend.js` (the spend-side analogue of `prove_mint_batch.js`, using the depth-20 PTAU produced by `scripts/snark/setup.sh`). `spend.circom`'s signal layout is exactly:

```
public noteRoot;      # an accepted root from the on-chain ring buffer
public nullifier;     # the nf from Step 2
public face;          # 1000e18 -- the BUCK to release (range-bounded to 128 bits)
```

```

public recipient;          # Bob's on-chain address (the payout destination)
public chainId;           # replay-domain separation

private flavor, v, rho,
    idHash, predicate;    # the Poseidon-5 opening
private pathElements[depth]; # depth-20 sibling hashes
private pathIndices[depth]; # left/right bit per level

```

The shipped circuit enforces five relation groups:

1. **Range bounds and value match.** Both the public face and the witness v pass through $\text{Num2Bits}(128)$, then $\text{face} == v$. This closes the field-overflow gap: v cannot wrap around r to claim more than its in-range share.
2. **Opening binds.** $\text{cm} = \text{Poseidon5}(\text{flavor}, v, \text{rho}, \text{idHash}, \text{predicate})$.
3. **Merkle membership.** $\text{MerkleProof}(20)$ walks $(\text{cm}, \text{pathElements}, \text{pathIndices})$ up the tree using $\text{Switcher} + \text{Poseidon}(2)$ per level and asserts the climb reaches noteRoot .
4. **Nullifier derivation.** $\text{nullifier} == \text{Poseidon3}(\text{rho}, \text{idHash}, 4242)$.
5. **Public-input ghost binding.** recipient and chainId each appear in a trivial $x \times x$ row. This is the minimal anchor needed for circom not to optimise out unreferenced public signals; once anchored, Groth16's IC[] commitment makes them non-malleable from the mempool's perspective.

Critically, cm is *not* a public input; it is recomputed inside the proof from the witness opening. The proof convinces the verifier that *some* cm in the tree satisfies (1)-(4); it does not reveal *which*.

This `spend.circom` note proof is *flavor-agnostic*: $A1$, $A2$, and $B1$ all use the same circuit and the same on-chain `spendVerifier.verifySpend` check for the cm -membership + nullifier discharge (nullifier tag 4242). Bob's wallet serializes the Groth16 proof into the standard `(uint256[2], uint256[2][2], uint256[2])` ABI tuple (256 bytes). The flavor-specific *identity* material – $eIss$ and the deposit-coupling proof dc ($A1/A2$), or $eDepForIss$ and the depositor-binding proof ($B1$), plus the G1-tie membership proof – is assembled separately and submitted alongside the note proof in Step 4.

10.5 Step 4: Submit Notes.spendCoupledA2 On Chain

Bob's note is $A2$ (private issuer), so he deposits via `spendCoupledA2`. Beyond the Step 1–3 note proof he supplies $eEnc$ – a *fresh re-encryption* of the leaf ciphertext $eIss$ under M_{rec} (same plaintext, new randomness, so the chain never sees the mint-time ciphertext again) – the deposit-coupling sigma dc (`alberta_buck.wallet.unilateral_a2.deposit_couple_prove`), the G1-tie membership proof `membershipProof`, and the note-binding proof `noteBindingProof` (`alberta_buck.wallet.note_binding`):

```

notes.spendCoupledA2(proofBytes, noteRoot, nullifier, 1000e18, bob,
    eEnc, dc, membershipProof, noteBindingProof);

```

The honest EVM walks `Notes.spendCoupledA2` (the function as it ships – a thin wrapper over the shared `_spendCoupled`):

```

require(address(identityRegistry) != address(0), "Notes: identity registry not set");
require(recipient != address(0), "Notes: zero recipient");
require(face > 0, "Notes: zero face");
require(!_isAcceptedRoot(root), "Notes: unknown root");

```

```

require(!nullifiers[nullifier], "Notes: already spent");

// (1) Note proof: cm in the pool tree under 'root', nullifier well-formed.
require(spendVerifier.verifySpend(
  proof, root, nullifier, face, recipient, block.chainid
), "Notes: bad spend proof");

// (2) Identity-M binding, half 1 -- the deposit-coupling sigma. msg.sender is
// Bob's deposit account; it proves the account is bound to m_rec AND that
// eEnc decrypts under m_rec to the point committed (blinded) in dc.P_I.
require(identityRegistry.verifyDepositCoupling(msg.sender, eEnc, dc),
  "Notes: bad deposit coupling");

nullifiers[nullifier] = true; // mark spent before paying out
noteFaceSum -= face;

// (3) Identity-M binding, half 2 -- membership of dc.P_I's point, bound to the
// SAME dc.P_I the coupling just constrained (the verifier derives its
// public-input limbs from dc.P_I, not from the proof bytes).
_verifyIdentityMembership(membershipProof, dc.P_I.X, dc.P_I.Y);

// (4) Identity-M binding, half 3 -- the note->eEnc re-encryption tie, binding
// eEnc to THIS note: the Groth16 proof's public inputs are derived from
// the caller's nullifier, eEnc, and dc.P_I.
_verifyNoteBinding(noteBindingProof, nullifier, eEnc, dc.P_I.X, dc.P_I.Y);

require(buck.transfer(recipient, face), // pool isCarrying => Carrying path
  "Notes: transfer failed");
emit SpentCoupledA2(nullifier, face, recipient, dc.P_I.X, dc.P_I.Y);

```

Six invariants are enforced in order:

a. **root** appears in the recent-roots window. A root that has rotated out is stale – Bob must rebuild the proof against a fresher root. b. **nullifier** has never been published before (Theorem 10.5 in the proofs says this single-bit check is sufficient for double-spend prevention). c. The note proof is valid against `SpendGroth16Verifier` via the `SpendVerifierAdapter`, which packs `[root, nullifier, face, recipient, chainid]` into the public-signal vector. By soundness Bob holds a Poseidon-5 opening of a `cm` under `root`, with `nullifier = Poseidon3(rho, idHash, tag)`. d. `verifyDepositCoupling(msg.sender, eEnc, dc)` succeeds: `msg.sender`'s registered account is bound to the identity scalar m_{rec} and `eEnc` decrypts under it to `dc.P_I` – an EIP-196 Okamoto sigma, all Identities hidden (M_{iss} is blinded in $P_I = M_{\text{iss}} + bH$). e. `_verifyIdentityMembership(membershipProof, dc.P_I.X, dc.P_I.Y)` succeeds: the G1-tie Groth16 proves `dc.P_I`'s underlying point is a member of the on-chain `identityRoot`. Because the contract feeds `dc.P_I` (not the proof bytes) as the public input, the membership is bound to *exactly* the point the coupling decrypted `eEnc` to – a colluding pair cannot key the ciphertext to a non-member and still spend. f. `_verifyNoteBinding(noteBindingProof, nullifier, eEnc, dc.P_I)` succeeds: the note-binding Groth16 (`circuits/note_binding.circom`) proves `eEnc` is a re-encryption, under M_{rec} , of the ciphertext committed in *this note's* `idHash` – the same `idHash` inside the nullifier gate (b) just consumed – and that its decryption is the same point `dc.P_I` commits. The adapter (`NoteBindingVerifierAdapter`) derives all 25 public inputs from the caller's `nullifier / eEnc / dc.P_I`, so the proof bytes carry only the Groth16 triple: an accept is necessarily about *this* spend. Without (f), gates (d)-(e) verify a ciphertext of the depositor's choosing; with it, only the addressed M_{rec} can spend, and an un-nameable A2 note stays un-spendable even under ciphertext substitution (*Proofs*, Theorem 12). Only after (a)-(f) does the contract pay out and emit `SpentCoupledA2(nullifier, face, recipient, P_I.X, P_I.Y)`.

If any of (a)-(f) reverts, the whole transaction reverts: no BUCK moves, the nullifier bit is never set, `noteFaceSum` is unchanged. **Bob gets one attempt per root window**; a failed spend retries with a refreshed proof, but never consumes a nullifier without paying out the BUCK.

An A1 note (public issuer) takes the identical path via `spendCoupledA1` – only `eIss`’s meaning differs (it encrypts M_{rec} under itself, so the membership certifies the *recipient*). A B1 bearer note takes `spendCoupledB1`, where the sigma is `verifyDepositorBinding` and P commits the depositor’s M_{dep} . All three are detailed in 11.

10.6 Step 5: BUCK ERC-20 Pays Out (the Carrying path)

The call inside `Notes.spendCoupledA2` is a plain `buck.transfer(bob, 1000e18)`. Because the `Notes` pool is bound `isCarrying` in the `IdentityRegistry`, BUCK routes that transfer through its internal `_carryingTransfer` path on this single egress. (`spendCoupledA1` and `spendCoupledB1` take the same path.) Nothing in `Buck.sol` is special-cased for the pool: it is just a registered, `isCarrying` account making an outbound transfer of 1,000 BUCK to a registered recipient. The result on-chain:

- `Buck.balanceOf(Notes)` decreases by exactly 1,000 BUCK – nominal balance only.
- `Buck.balanceOf(Bob)` increases by exactly 1,000 BUCK.
- `Buck.Transfer(Notes, Bob, 1000e18)` is emitted (a standard ERC-20 event; the carrying apportionment is internal, with no distinct event).
- `Notes.Spent(nullifier, 1000e18, Bob)` is emitted.
- `Notes.nullifiers[nullifier]` becomes `true`.

What "carrying" means in the ledger. BUCK demurrage is a balance-time integral: each account stores its `buckSeconds` (the crystallised $\int B dt$) plus the `timestamp` of its last touch. A transfer *from a non-Carrying account* settles the sender’s accrued fee (a one-shot burn) and hands the recipient fresh, zero-age BUCK. A transfer from an `isCarrying` account – the `_carryingTransfer` path – differs in two surgical ways:

1. **No sender-side fee burn.** The pool sheds a proportional slice of its live buck-seconds rather than burning a fee; its per-BUCK age is preserved.
2. **Recipient absorbs the pool’s age.** The recipient inherits that slice of buck-seconds (`carried = liveBs * amount / poolRaw`), added to its own:

```
liveBs    = pool.buckSeconds + pool.raw * elapsed;
carried   = liveBs * amount / pool.raw;
pool.buckSeconds    = liveBs - carried;    // pool sheds its share
recipient.buckSeconds += carried;          // recipient inherits the age
```

System-wide BUCK-age is preserved across the call (no fee is burned, no fresh-BUCK dilution at the recipient); the demurrage that the pool would have burned on a standard outgoing transfer instead rides into Bob’s account as inherited buck-seconds. Bob will pay it the next time he makes a standard outgoing transfer.

Why this is the right rule for the pool. Per-note age is opaque – the pool holds aggregate BUCK behind opaque commitments and cannot tell Bob’s note’s age from any other note’s. The Carrying path collapses the calculation to the pool’s average per-BUCK age at spend time: every spender of the same `face` absorbs the same buck-seconds slice, keeping the spend-time anonymity set intact. The trade-off is that Bob inherits the pool’s average age regardless of how long *his specific*

note sat in the tree. Anyone whose note rested longer than the pool average comes out ahead; this is the cost of anonymity, and the upside for long-term holders.

The full discussion (one new BUCK method, one new event, no per-account mode flag) is in **Demurrage-Carrying Transfers** below.

10.7 What the Observer Sees at Spend (Mallory Returns)

After `T_spend` confirms, Mallory can read:

```
T_spend:
  from: Bob (a registered IdentityRegistry account)
  to:   Notes (system-public)
  data: Notes.spendCoupledA2(proofBytes, noteRoot, nullifier, 1000e18, Bob,
                             eIss, dc, membershipProof)

events:
  Buck.Transfer(Notes, Bob, 1000e18)
  Notes.SpentCoupledA2(nullifier, 1000e18, Bob, P_I.x, P_I.y)
```

What she infers:

- **That a note was spent.** Yes – the nullifier is fresh and the pool balance dropped by 1,000 BUCK.
- **That the spend was for 1,000 BUCK to Bob.** Yes – `face` and `recipient` are public inputs (they must be, to tell the ERC-20 whom to pay).
- *Which commitment in the tree was consumed?* **No** – `cm` is a private witness to the SNARK. Mallory's anonymity set is every unspent commitment of flavor `A` in the pool whose hidden `v` could equal 1,000 BUCK. At deployment scale this is potentially the entire pool minus prior nullified entries; in a same-denomination regime (many 1,000-BUCK notes in circulation) it is exactly the 1,000-BUCK subset.
- *Which mint did the spent note come from?* **No** – there is no cross-reference to `T_mint` in `T_spend`. The only link back to any specific mint would be the nullifier, but nullifiers are Poseidon-PRF outputs over secret material; inverting one to recover (`flavor`, `M_rec`, `rho`) is exactly the pre-image resistance we assume.
- *Who held the note before Bob?* **No**. If Alice gave the note to Carol who handed it to Dave who sold it to Bob, none of those transfers touched the chain. The chain sees only "at mint Alice deposited, later Bob withdrew" – the intermediate holders are invisible.

The residual linkability surfaces are (i) **face** (the revealed amount) and (ii) **recipient** (the on-chain payout address). Both are unavoidable at the SNARK/ERC-20 boundary – the contract must know where and how much to pay. The community-standardized mitigation is fixed-denomination notes (so revealing **face** does not subdivide the anonymity set) and batched, relay-mediated payouts (so revealing **recipient** need not reveal the *true* beneficiary). Both are orthogonal to the core spend circuit and can be layered on top.

10.8 Why Alice Cannot Steal Bob's Note (Even Between Rooms)

Can *any* actor other than Bob – Alice-the-issuer, a prior holder, or a random Mallory – redeem `cm_0`? Three protections apply, the third being the Identity-M deposit binding (the deposit-coupling `sigma` + the bound membership):

1. **Opening secrecy.** `spend.circom` requires the full Poseidon-5 opening (`flavor`, `v`, `rho`, `idHash`, `predicate`) as a private witness. Without `rho_0` and `idHash_0` the circuit is unsatisfiable, so no one without the opening can produce a valid nullifier or proof. For Alice’s secrets, this is the load-bearing protection on bearer-flavor (B1) notes: she could in principle race the bearer to deposit, which is the civil-trust semantics of bearer cash and is discussed at length in `alberta-buck-notes.org §Long-Term Holding Safety`.
2. **Recipient-binding at the payout site.** `recipient` is a public SNARK input ghost-bound by `recipient * recipient`, so Groth16’s IC[[commitment makes it non-malleable. A front-runner who intercepts Bob’s `proofBytes` and resubmits with a different `recipient` is rejected by the verifier. `chainId` is bound identically, blocking cross-chain replay.
3. **Identity binding via the deposit-coupling sigma + bound membership.** For an A2 deposit Bob provides `eIss` and the coupling proof `dc`; `spendCoupledA2` calls `verifyDepositCoupling(msg.serialize(eIss, dc))`, which reads Bob’s registered (`pk_dep`, `E_dep`) from storage and confirms a single `sk_dep` and identity scalar m_{rec} bind the account ($C_d == m_{rec} * G + sk_{dep} * R_d$) and decrypt `eIss` under m_{rec} to the committed point `dc.P_I`. The G1-tie membership then proves `dc.P_I`’s underlying point is in the on-chain `identityRoot`, bound to the *same* `dc.P_I` the coupling fixed. Alice-the-issuer cannot redeem to herself even knowing `idHash_0` and `rho_0`: she is not bound to m_{rec} , so the coupling fails. The sigma costs $\sim 36K$ gas (EIP-196 `ecMul/ecAdd` + `keccak` Fiat-Shamir); the membership Groth16 verify is $\sim 350K$, constant.

The strict identity-binding guarantee is now a chain-observable property: the `verifyDepositCoupling` + membership booleans are the gate. The note proof enforces the `cm-membership` + nullifier discharge; the coupling + membership enforce that the depositor is the registered Identity the note named – all A1/A2 identity, security, and privacy properties preserved (and, for A2, the issuer is provably nameable – the collusion gap closed).

10.9 B1 Bearer Notes

A B1 note (bearer, public issuer) is redeemed the same way, with the depositor binding playing the role the coupling does for A2: `spendCoupledB1` calls `verifyDepositorBinding` (the depositor re-encrypts its own M_{dep} under the public issuer’s key, coupled to its payout account) plus a membership proof of M_{dep} . Knowledge of the Poseidon-5 opening is still the *bearer* authorisation – if two parties hold copies of the same B1 opening (a paper note photographed and forwarded), whichever submits first wins, the intended civil-trust semantics of bearer cash – but the depositor is now additionally proven to be a registered Identity the issuer can name (see 11).

10.10 Recapping Property C (Only Valid Notes Redeem)

The original Property C question – *subsequently only those N valid notes can be deposited* – becomes concrete at this layer. Claim: no actor can redeem a commitment `cm` unless

a. `cm` was appended to `commitments[]` by a prior `Notes.mint` transaction whose Groth16 proof verified (i.e., `cm` is part of an honest mint batch), and b. The actor holds the opening of `cm`, and c. For A-spends, the actor is the depositor keyed by `M_rec` in that opening.

Proof sketch. The spend circuit’s relation 2 (Merkle membership) rejects any `cm` not in `noteRoot`. The mint circuit’s relations 1-2 (opening binding and sum conservation) are what places a `cm` in `noteRoot` in the first place, and by the mint-side soundness story in **Why Alice Cannot Cheat** every such `cm` corresponds to a well-formed opening whose `v` is included in the mint’s `totalFace`

escrow. The spend circuit’s relation 5 (`face == v`) then ensures the BUCK released equals exactly the share of the escrow that `cm` was allocated. Summing over every accepted spend of a given mint batch, the cumulative BUCK released equals at most the `totalFace` escrowed – property C follows from mint-side value conservation plus spend-side nullifier uniqueness.

The earlier caveat about a missing `Num2Bits` range check on `v[i]` at mint time is closed: `mint_batch.circom` range-bounds each `v[i]` to 128 bits and `spend.circom` range-bounds both `face` and the witness `v` identically, so an integer overshoot of `totalFace` by a multiple of `r` is structurally impossible at either end. Property C now rests on those range bounds plus the nullifier-uniqueness check and Merkle membership.

DRAFT

11 The Identity-M Spend Path (A1, A2, B1)

Step 4 above showed the A2 deposit in the redemption walk; this section gives the per-flavour detail for all three. The spend is *one gadget*, parameterised by flavour: an EIP-196 Okamoto sigma producing a single committed point P , plus a Poseidon-Merkle membership proof *bound to that same P* , directly enforcing "the counterparty Identity is a registered KYC Identity" at spend time – without the circuit overhead of non-native $\mathbb{G}_1$ arithmetic. Three entrypoints instantiate it:

- **A2 (addressed, private issuer)** – `Notes.spendCoupledA2`. The recipient deposits via `verifyDepositCoupling`, proving its account is bound to the M_{rec} that decrypts the note ciphertext $eIss$ to the issuer Identity committed in P_I ; the membership certifies that issuer is registered – closing the A2 recipient-key collusion gap.
- **A1 (addressed, public issuer)** – `Notes.spendCoupledA1`. Identical on-chain path; the note encrypts M_{rec} under itself, so P_I commits the *recipient* and the membership certifies *them*. The issuer is public, named off chain at mint (the batch Schnorr).
- **B1 (bearer, public issuer)** – `Notes.spendCoupledB1`. The dual: the depositor re-encrypts its own M_{dep} under the public issuer's key (`verifyDepositorBinding`), coupled to its payout account and to a committed P_{dep} ; the membership certifies M_{dep} , and the issuer decrypts the event to name the depositor.

All three gate on the *membership* of the committed point in the registry-Identity accumulator rt_{id} , whose root is the single on-chain `IdentityRegistry.identityRoot`, updated incrementally by each `register()` / `bindContract()`. In every case the membership is *bound* to the sigma's own P – the verifier derives its public inputs from P , not from the prover's bytes, so the two halves cannot be decoupled.

The *addressed* flavours (A1, A2) add a fourth gate: the note $\leftrightarrow eEnc$ re-encryption tie (`circuits/note_binding` behind `INoteBindingVerifier`). The flavour-agnostic note proof exposes no *idHash*, so without the tie nothing binds the deposit-coupling ciphertext to the *specific note being spent* – a holder of a stolen opening could supply a self-addressed *eEnc*, and an A2 coalition could substitute a nameable one. The tie proves, in one Groth16 proof, that the spend's *eEnc* re-encrypts the ciphertext committed in the *idHash* behind *this spend's nullifier*, and that its decryption is the very point P_I the sigma committed (*Proofs*, Theorem 12).

11.1 The Shared Primitives

Before the flavour-specific walkthroughs, three primitives are common to all three:

- **The identity accumulator.** Identity points $M = mG$ for every registrant are folded into a Poseidon Merkle tree whose root rt_{id} lives on chain as `IdentityRegistry.identityRoot`, updated incrementally by each `register()` / `bindContract()` (the Tornado-style `filledSubtrees` pattern, depth 10). A newly issued M is usable only after its insertion – the *commit-before-use* discipline from *alberta-buck-notes-identity-axis*. The one canonical tree is `alberta_buck.registry.tree.IdentityTree` (the wallet and the sim share it; `alberta_buck.wallet.unilateral_a2.IdentityTree` is a thin point-centric facade over it), matched byte-for-byte by the contract.
- **The membership-plus-G1-tie SNARK (shipped and bound).** A single Groth16 circuit, `circuits/identity_membership_g1tie.circom` (verified by `IdentityMembershipG1TieVerifier.sol`, ~350K gas, constant), proves both halves at once: $\ell(M) = \text{Poseidon}(M.x, M.y) \in \text{rt}_{\text{id}}$ AND

the G1 tie $P_I = M + bH$ (one BN254 G_1 point addition via `circom-lib's EllipticCurveAddOptimised`). Its nine public inputs are $[rt_{id}, PI_x[0..3], PI_y[0..3]]$ – the committed point P_I as 64-bit $\mathbb{F}_q$ limbs. **The binding:** the on-chain verifier derives those limbs from the deposit-coupling sigma's *own* P_I , not from prover-supplied bytes, so a Groth16 accept proves membership of *exactly* the point the sigma decrypted $eIss$ to. A colluding pair cannot answer the coupling with one P_I and the membership with another (`IdentityMembershipBinding.t.sol`, 5 tests).

- **The aggregator.** Multiple registries and feature authorities feed sub-roots into a `CentralMerkleService` (depth 10, up to 2^{10} sub-trees), whose single aggregated root is the on-chain `identityRoot`. Prototyped in `alberta_buck.registry.merkle_service` (49 tests).

11.2 A2 Identity-Targeted Spend (The Unilateral Receipt Flow)

Bob's note is flavour A2 (addressed, private issuer). The note opening carries $eIss = (R_e, C_e)$ – the issuer's Identity M_I encrypted under Bob's Identity point M_{rec} , plus the anti-framing binding (`verifyIssuerReenc`) that ties $eIss$ to the issuer's own registered Identity. Bob holds the one secret m_{rec} that can decrypt it.

==== Step A0: Bob Holds the A2 Note ====

The artifact Bob received off chain:

```
note = (
  flavor = A2, v = 1000e18, rho, idHash, predicate, # the commitment opening
  eIss = (R_e, C_e),                               # M_I under M_rec
  binding,                                           # verifyIssuerReenc
  mint_tx_ref,                                       # where cm was folded
)
```

==== Step A1: Bob Chooses a Deposit Account ====

Bob selects *any* registered Fountain account bound to his identity scalar m_{rec} . His wallet reads the account's registered $(pk_{dep}, E_{dep} = (R_d, C_d))$ from the IdentityRegistry. Because all accounts share the same m_{rec} , he is free to use a fresh account every time – no re-registration, no key pinning.

==== Step A2: Generate the Deposit-Coupling Proof ====

Bob's wallet builds the *deposit-coupling proof* (`deposit_couple_prove` in `alberta_buck.wallet.unilateral_a`). The prover proves knowledge of (m_{rec}, sk_{dep}, b) such that:

$$\begin{array}{lll}
\text{(E4)} & pk_{dep} = sk_{dep} G & \text{– the real account key;} \\
\text{(E2)} & C_d = m_{rec} G + sk_{dep} R_d & \text{– the account is bound to } M_{rec} = m_{rec} G; \\
\text{(E3)} & C_e - P_I = m_{rec} R_e - b H & \text{– } eIss \text{ decrypts under } m_{rec} \text{ to } P_I - bH.
\end{array}$$

This is a 3-witness Okamoto sigma over EIP-196, producing 8 struct members: $(e, s_m, s_s, s_b, A2, A3, A4, P_I)$. Zero identities are revealed; the issuer Identity M_I is perfectly hidden in $P_I = M_I + bH$.

==== Step A3: Generate the Identity Membership Proof ====

The wallet generates the G1-tie Groth16 proof (`circuits/identity_membership_g1tie.circom`). Its *public* inputs are the on-chain `identityRoot` and the committed point $P_I = M_I + bH$ – the *same* P_I the deposit-coupling sigma carries – as 64-bit $\mathbb{F}_q$ limbs. Its *private* witnesses are the issuer Identity M_I , the blind $T = bH$, and the Merkle path. The circuit proves, in one proof,

$$P_I = M_I + T \quad \wedge \quad \ell(M_I) = \text{Poseidon}(M_I.x, M_I.y) \in rt_{id},$$

i.e. *the point the sigma decrypted to is a genuine, registered KYC Identity*. M_I never appears – only the blinded P_I is public, so the issuer stays hidden from Mallory while being provably a member.

The wallet does not put P_I into the proof *bytes* it submits; only the Groth16 triple (a, b, c) travels. The contract supplies P_I to the verifier from the deposit-coupling sigma (Step A2), which is what *binds* the two halves – see Step A5.

==== Step A4: Submit the Deposit ====

Bob submits the identity-M-bound deposit entrypoint, `Notes.spendCoupledA2`:

```
notes.spendCoupledA2(
    proof,                // Groth16: note opening + noteRoot membership
    noteRoot,
    nullifier,            // Poseidon(rho, idHash, A-tag)
    face,                // 1000e18
    recipient,           // Bob's payout address
    eIss,                // (R_e, C_e): the leaf ciphertext
    dc,                  // DepositCouplingProof (e,s_m,s_s,s_b,A2,A3,A4,P_I)
    membershipProof,     // G1-tie Groth16 (a,b,c) bytes; P_I supplied by the contract
);
```

`eIss`, `dc`, and `membershipProof` are the identity-M additions. Crucially, `membershipProof` carries *only* the Groth16 triple; the committed point P_I it is checked against is read by the contract from `dc.P_I` – the wallet cannot decouple them.

==== Step A5: On-Chain Verification (the binding) ====

`spendCoupledA2` gates on five conditions, reading the committed point `dc.P_I` *once* and feeding it to *both* identity halves:

1. Standard note spend: `noteRoot` is accepted, `nullifier` is fresh, the Poseidon-5 opening Groth16 proof verifies (`spendVerifier.verifySpend`).
2. Deposit coupling: `verifyDepositCoupling(msg.sender, eIss, dc)` accepts – the EIP-196 sigma confirms the depositing account is bound to M_{rec} and `eIss` decrypts under it to `dc.P_I`. (7 Solidity + 11 Python tests.)
3. Bound identity membership: `verifyMembership(membershipProof, identityRoot, dc.P_I.X, dc.P_I.Y)` accepts – the G1-tie Groth16 proves `dc.P_I`'s underlying M_I is a registered KYC Identity. The verifier derives the circuit's P_I limbs from `dc.P_I`, *not* from the proof bytes, so this is provably the *same* point gate 2 constrained.
4. BUCK transfer: pool pays `face` to `recipient` via the Carrying path.
5. Nullifier recorded: `nullifiers>nullifier] = true`; the contract emits `SpentCoupledA2(nullifier, face, recipient, P_I.X, P_I.Y)`.

If any check fails, the whole transaction reverts. No BUCK moves, no nullifier is consumed. (End-to-end: `NotesCoupledA2.t.sol`, 8 tests.)

Why this closes the collusion gap. Suppose a colluding issuer keys `eIss` to a throwaway point so the recipient "cannot" name the issuer. Gate 2 still passes (the depositor can prove `eIss` decrypts to *some* point P_I under their m_{rec}). But that point's underlying M_I is then *not* a registered Identity, so *no* G1-tie proof exists for it, and gate 3 fails: **the deposit reverts**. There is no spendable-but-unnameable note. An honest note, by contrast, decrypts to the issuer's registered M_I , a member, and spends cleanly.

==== Step A6: The Unilateral Receipt ====

After the spend confirms, Bob’s wallet produces a *unilateral receipt* naming both parties – from m_{rec} alone:

$$\begin{aligned} M_I &= C_e - m_{\text{rec}} R_e && \text{(the issuer Identity);} \\ M_{\text{rec}} &= m_{\text{rec}} G && \text{(the recipient’s own).} \end{aligned}$$

A *verifiable decryption* proof (`verifiable_decrypt_prove`, a Chaum-Pedersen DLEQ) makes the receipt third-party-checkable without revealing m_{rec} . The receipt tuple:

$$\mathcal{R} = (M_I, M_{\text{rec}}, v, eIss, \text{vd}, \text{binding}, \text{issuer addr}, \text{chainid}).$$

A court or auditor checks four links (`verify_receipt`):

#	Check	Establishes
1	<code>verifyIssuerReenc</code> vs. issuer’s on-chain record	$eIss$ re-encrypts the issuer’s <i>registered</i> M_I (no framing)
2	<code>verifiable_decrypt_verify</code> ($eIss$, M_{rec} , M_I)	$eIss$ decrypts under M_{rec} to M_I
3	$\ell(M_I) \in \text{rt}_{\text{id}}$	the issuer Identity is registered (the <i>coupling</i>)
4	$\ell(M_{\text{rec}}) \in \text{rt}_{\text{id}}$	the recipient Identity is registered

Full specification and tests: *alberta-buck-notes-unilateral*.

11.3 A2 in Code: The Full Cycle, End to End

The cryptography above is small enough to run. This section is a complete, *executing* transcript against the shipped wallet (`alberta_buck.wallet.unilateral_a2` over BN254 $\mathbb{G}_1$) – mirroring `test_unilateral_a2.py`. The blocks below are org-babel source blocks sharing one Python session: re-exporting this document re-runs them top to bottom, and the printed **RESULTS** are regenerated. The session is seeded, so a re-run reproduces this transcript byte for byte – and fails loudly if the wallet API drifts.

The cast: two Identities, one of them with two accounts. An *Identity* is a KYC-bound scalar m and its point $M = mG$. The Identity Fountain lets one m back unlimited unlinkable *accounts*, each an ElGamal encryption $E_{\text{addr}} = (R, C) = (rG, M + rpk)$ of the *same* M under that account’s own key.

```
import random
from alberta_buck.wallet.bn254 import G1, ORDER, mul, add, neg, eq, rand_scalar
from alberta_buck.wallet.elgamal import elgamal_encrypt, elgamal_decrypt
from alberta_buck.wallet.unilateral_a2 import (
    IdentityTree, mint_unilateral_a2,
    deposit_couple_prove, deposit_couple_verify,
    make_receipt, verify_receipt,
)

_seed = random.Random(0xA1BE27AB0CC)      # ONE deterministic transcript
rng = lambda: _seed.getrandbits(256)

CHAINID = 1
def account(m):
    # one Fountain account bound to identity m
    sk = rand_scalar(rng); pk = mul(G1, sk)
    M = mul(G1, m)                # the shared Identity point
    E = elgamal_encrypt(M, pk, rand_scalar(rng))    # registered credential
    return dict(m=m, M=M, sk=sk, pk=pk, E=E)
```

```

def pt(P):
    # compact point display
    return f"({int(P[0]) % 10**8:>8d}..., {int(P[1]) % 10**8:>8d}...)"

m_iss, m_rec = rand_scalar(rng), rand_scalar(rng)
issuer = account(m_iss) # the payer (private issuer)
rec0 = account(m_rec) # recipient account #0
rec1 = account(m_rec) # recipient account #1 (SAME identity)
M_rec = mul(G1, m_rec) # what the issuer learns out of band

tree = IdentityTree() # the registry-Identity accumulator
for a in (issuer, rec0): # one leaf per *identity*
    tree.insert(a["M"])

print(f"issuer Identity M_I = {pt(issuer['M'])}")
print(f"recipient Identity M_rec = {pt(M_rec)} (accounts rec0, rec1)")
print(f"identity tree root = {hex(tree.root())[:20]}...")

issuer Identity M_I = (34794633..., 55370756...)
recipient Identity M_rec = (78777287..., 1501867...) (accounts rec0, rec1)
identity tree root = 0x1b94c62584dc6075ad...

```

Mint: encrypt the issuer's own Identity under the recipient's point. The issuer addresses the note to *the Identity* M_{rec} , not an account. It encrypts its *own* registered M_I under M_{rec} used as a public key; the anti-framing binding (`verifyIssuerReenc`) proves $eIss$ re-encrypts the issuer's OWN registered M_I – it cannot name a victim.

```

note = mint_unilateral_a2(issuer["sk"], issuer["E"], M_rec,
                          v=1000, rho=rand_scalar(rng),
                          issuer=0xA11CE, chainid=CHAINID, rng=rng)
assert eq(note.M_I, issuer["M"])
print(f"minted A2 note: v = {note.opening.v}")
print(f"cm = {hex(note.cm)[:20]}... (the only on-chain artifact)")
print(f"idHash = {hex(note.idHash)[:20]}... (Poseidon8(eNote, eIss))")
print(f"eIss = ElGamal(M_I under M_rec) R = {pt(note.eIss.R)}")

```

```

minted A2 note: v = 1000
cm = 0x119e5f967cfb3426ea... (the only on-chain artifact)
idHash = 0x5c689616da10e765ed... (Poseidon8(eNote, eIss))
eIss = ElGamal(M_I under M_rec) R = (24688725..., 93913945...)

```

Delivery: the recipient names the issuer with one secret. The decryption secret is m_{rec} – shared by *every* account the recipient owns – because $M_{\text{rec}} = m_{\text{rec}}G$:

$$C_e - m_{\text{rec}}R_e = (M_I + r'M_{\text{rec}}) - m_{\text{rec}}(r'G) = M_I.$$

```

M_I = elgamal_decrypt(note.eIss, m_rec)
assert eq(M_I, issuer["M"])
print("recipient decrypts eIss with m_rec -> the issuer's registered Identity:")
print(f"C_e - m_rec*R_e = {pt(M_I)} == M_I")

```

```

recipient decrypts eIss with m_rec -> the issuer's registered Identity:
C_e - m_rec*R_e = (34794633..., 55370756...) == M_I

```

Deposit from ANY account bound to m_{rec} . The issuer never knew which account would cash the note. Both `rec0` and `rec1` can, because both share m_{rec} . The blind b is drawn once here because the note-binding SNARK below must commit the *same* $P_I = M_I + bH$ the sigma publishes.

```

b = rand_scalar(rng)          # the P_I blind -- shared with the SNARK below
for name, acct in (("rec0", rec0), ("rec1", rec1)):
    dc = deposit_couple_prove(m_rec, acct["sk"], acct["E"], note.eIss,
                             account=0xB0B, chainid=CHAINID, b=b, rng=rng)
    ok = deposit_couple_verify(acct["pk"], acct["E"], note.eIss, dc,
                              account=0xB0B, chainid=CHAINID)
    print(f"deposit-coupling sigma from {name}: verifies = {ok} "
          f"P_I = {pt(dc.P_I)}")

```

```

deposit-coupling sigma from rec0: verifies = True  P_I = (74064912..., 52659236...)
deposit-coupling sigma from rec1: verifies = True  P_I = (74064912..., 52659236...)

```

dc is the EIP-196 Okamoto sigma ($E4/E2/E3$ of Step A2). Its only Identity-derived public value is $P_I = M_I + bH$, *perfectly hidden* by the fresh blind b . On chain, `Notes.spendCoupledA2` hands this same `dc.P_I` to the membership verifier *and* to the note-binding verifier – the binding:

```

// inside Notes.spendCoupledA2 (abridged):
require(reg.verifyDepositCoupling(msg.sender, eEnc, dc), "bad deposit coupling");
//                                     dc.P_I --- the same point ---v
_verifyIdentityMembership(membershipProof, dc.P_I.X, dc.P_I.Y);
_verifyNoteBinding(noteBindingProof, nullifier, eEnc, dc.P_I.X, dc.P_I.Y);

```

The note-binding witness: tie $eEnc$ to THIS note. The spend supplies $eEnc$, a *fresh re-encryption* of the leaf’s $eIss$ (same plaintext, new randomness s) – the chain never sees the mint-time ciphertext again. The `note_binding` SNARK proves $eEnc$ re-encrypts the ciphertext committed in the *idHash* behind this spend’s nullifier, and that its decryption is the P_I point the sigma committed. The witness builder runs the whole relation in Python (every constraint is asserted before anything is proved):

```

from alberta_buck.wallet.note_binding import make_note_binding_witness

s = rand_scalar(rng)          # fresh re-encryption randomness
witness = make_note_binding_witness(
    rho=note.opening.rho, eNote=note.eNote, eIssCommitted=note.eIss,
    s=s, m_rec=m_rec, b=b, M_I=note.M_I, r_iss=note.r_prime,
)
print(f"binding witness built: {len(witness)} signals, all relations asserted")
print(f" public nullifier = {hex(int(witness['nullifier']))[:20]}...")
print(f" eEnc.R limb0    = {witness['eEncRx'][0]} (fresh -- not the leaf's eIss.R)")
R0x_leaf = witness["R0_limb"][0][0]
print(f" eIss.R limb0    = {R0x_leaf}")
assert witness["eEncRx"][0] != R0x_leaf # re-encryption, not reuse

binding witness built: 20 signals, all relations asserted
public nullifier = 0x315cd0f43dcbfc5235...
eEnc.R limb0    = 7194077872496329669 (fresh -- not the leaf's eIss.R)
eIss.R limb0    = 13585057060835952469

```

Proving and verifying this witness is the heavyweight step – a 2.4M-constraint Groth16 circuit – so it runs in the build pipeline rather than in this document’s session. The real transcript (Apple Silicon, `make nix-snark-note-binding`):

```

$ make nix-snark-note-binding
--- Compiling circuit (circom --c --no_asm --02) ---
    R1CS: 471M    (2.4M constraints, 0 linear after --02, 25 public inputs)
--- Building C++ witness generator ---              (~9 min, gcc -03 -fno-strict-aliasing)
--- Generating witness ---                          (~9 s, 73 MB witness.wtns)

```

```

--- Checking witness (snarkjs wtns check) ---
[INFO]  snarkJS: WITNESS IS CORRECT
--- Groth16 setup (ptau: pot22_final.ptau) ---    (1.3 GB zkey)
--- Proving (rapidsnark) ---                      (seconds, vs ~33 s for snarkjs)
[INFO]  snarkJS: OK!
Result: Valid proof                                (rapidsnark verifier agrees)
  -> src/NoteBindingGroth16Verifier.sol (EIP-197 proof-B swap applied)
  -> test/vectors/note_binding/proof.json
$ forge test --match-contract NoteBindingVerifier
Suite result: ok. 9 passed; 0 failed                (~501K gas raw, ~508K via adapter)

```

The unilateral receipt: the recipient names BOTH parties, alone. From m_{rec} *alone* – no issuer cooperation – the recipient recovers both Identities and produces a third-party-checkable receipt:

```

receipt = make_receipt(m_rec, note, issuer=0xA11CE, chainid=CHAINID,
                      tree=tree, rng=rng)
res = verify_receipt(receipt, issuer["pk"], issuer["E"], tree.root(), tree)
assert res.valid
assert eq(res.issuer_M, issuer["M"])      # the payer, named
assert eq(res.recipient_M, M_rec)         # the payee, named
assert res.value == 1000
print(f"unilateral receipt: {res.reason}")
print(f"  names issuer      {pt(res.issuer_M)}")
print(f"  names recipient  {pt(res.recipient_M)}")
print(f"  value            {res.value}")

unilateral receipt: VALID
  names issuer      (34794633..., 55370756...)
  names recipient  (78777287..., 1501867...)
  value            1000

```

Collusion fails twice: un-nameable is un-spendable, and substitution cannot bind. First the classic collusion: an issuer keys $eIss$ to a *throwaway* point. The mint anti-framing binding still accepts (it only forces $eIss$ over the issuer's own M_I), but the recovered point is not a tree member – the receipt is INVALID and the membership proof cannot exist. Second, the substitution escape the note-binding closes: the colluders *can* build a binding witness over their bogus ciphertext (they know its randomness), but its $idHash$ is not the spent note's – the SNARK's nullifier disagrees with the one the spend consumed, and the on-chain public-input derivation rejects the proof.

```

pk_bogus = mul(G1, rand_scalar(rng))          # a key the recipient lacks
r_bogus  = rand_scalar(rng)                   # colluders know their own r
eIss_bogus = elgamal_encrypt(issuer["M"], pk_bogus, r_bogus)
M_I_bogus = elgamal_decrypt(eIss_bogus, m_rec) # garbage, not issuer["M"]
print(f"colluding mint: eIss keyed to a throwaway -- decrypts to {pt(M_I_bogus)}")
print(f"  registered identity? {tree.contains(M_I_bogus)} -> membership unprovable")

b_bogus = rand_scalar(rng)
witness_sub = make_note_binding_witness(
  rho=note.opening.rho, eNote=note.eNote, eIssCommitted=eIss_bogus,
  s=rand_scalar(rng), m_rec=m_rec, b=b_bogus, M_I=M_I_bogus, r_iss=r_bogus,
)
print(f"substitute-ciphertext nullifier = {hex(int(witness_sub['nullifier']))[:20]}...")
print(f"the spent note's nullifier      = {hex(int(witness['nullifier']))[:20]}...")
assert witness_sub["nullifier"] != witness["nullifier"]
print("  -> public-input mismatch: the substitute proof cannot bind THIS spend")

```

```

colluding mint: eIss keyed to a throwaway -- decrypts to (27018772..., 43291458...)
  registered identity? False -> membership unprovable
substitute-ciphertext nullifier = 0x271cc47267f3952333...
the spent note's nullifier      = 0x315cd0f43dcbfc5235...
-> public-input mismatch: the substitute proof cannot bind THIS spend

```

That is the whole guarantee in one screen: *the recipient holds the single secret that names both parties, and the issuer cannot make a spendable note that the recipient is unable to name – nor can any holder bind a different ciphertext to it.*

11.4 A1 in Code: The Public-Issuer Addressed Cycle

A1 is A2 with the issuer in the clear: *idHash* commits the issuer's public Identity and batch Schnorr instead of an encrypted *eIss*, and the leaf ciphertext *eRec* encrypts M_{rec} *under itself* – so the deposit coupling commits the *recipient's* Identity and the membership certifies *them*. (The batch Schnorr is checked at the mint transaction level – `verifyIssuerSchnorr`; here it is a synthetic signature, since the session has no chain.)

```

from alberta_buck.wallet.unilateral_a1 import (
    mint_unilateral_a1, make_receipt_a1, verify_receipt_a1,
)

k = rand_scalar(rng)
sigma_R, sigma_s = mul(G1, k), (k + rand_scalar(rng) * rand_scalar(rng)) % ORDER

note_a1 = mint_unilateral_a1(M_rec, v=100, rho=rand_scalar(rng),
                             m_issuer=m_iss, sigma_R=sigma_R, sigma_s=sigma_s,
                             rng=rng)
print(f"minted A1 note: v = {note_a1.opening.v}")
print(f"  idHash = Poseidon8(eNote, m_issuer, sigma) = {hex(note_a1.idHash)[:20]}...")
print(f"  eRec   = ElGamal(M_rec under M_rec)      R = {pt(note_a1.eRec.R)}")

rcpt_a1 = make_receipt_a1(m_rec, note_a1, issuer["M"], issuer=0xA11CE,
                          chainid=CHAINID, tree=tree, rng=rng)
res_a1 = verify_receipt_a1(rcpt_a1, tree.root(), tree)
assert res_a1.valid and res_a1.value == 100
print(f"A1 receipt: {res_a1.reason} -- issuer public, recipient proves the note")
print(f"  addressed THEIR identity: eRec decrypts under m_rec to M_rec")

minted A1 note: v = 100
  idHash = Poseidon8(eNote, m_issuer, sigma) = 0x2ff6525153c4aa14df...
  eRec   = ElGamal(M_rec under M_rec)      R = (98448865..., 79624823...)
A1 receipt: VALID -- issuer public, recipient proves the note
  addressed THEIR identity: eRec decrypts under m_rec to M_rec

```

11.5 B1 Bearer Spend with Depositor Naming (The A2 Dual)

Carol holds a B1 note from a public issuer. She has never been identified to the issuer; at spend she reveals her Identity to the issuer *alone*, through an encryption only the issuer can decrypt.

==== Step B0: Carol Holds a B1 Note ====

Carol's artifact carries the issuer's public Identity M_{issuer} in the clear with a Schnorr signature, but *no* per-recipient ciphertext (bearer semantics).

==== Step B1: Carol Encrypts Her Identity for the Issuer ====

At deposit Carol's wallet encrypts her Identity under the public issuer's key:

$$E_{\text{dep} \rightarrow \text{iss}} = (R_f, C_f) = (rG, M_{\text{Carol}} + rpk_{\text{iss}}).$$

Only sk_{iss} recovers M_{Carol} ; to Mallory the ciphertext is IND-CPA noise.

==== Step B2: Generate the Depositor-Binding Proof (A2 dual) ====

Carol's wallet proves (`b1_bind_prove`, `verifyDepositorBinding`) knowledge of $(m_{\text{Carol}}, sk_{\text{dep}}, r)$ such that the ciphertext encrypts the *same* Identity bound to Carol's payout account:

$$\begin{aligned}
 (\text{E4}) \quad & pk_{\text{dep}} = sk_{\text{dep}} G; \\
 (\text{E2}) \quad & C_d = m_{\text{Carol}} G + sk_{\text{dep}} R_d; \\
 (\text{F1}) \quad & R_f = r G; \\
 (\text{F2}) \quad & C_f = m_{\text{Carol}} G + r pk_{\text{iss}}; \\
 (\text{P}) \quad & P_{\text{dep}} = m_{\text{Carol}} G + b H.
 \end{aligned}$$

The shared witness m_{Carol} couples (E2), (F2), and (P): the Identity bound to Carol's payout account is exactly the one encrypted for the issuer *and* the one committed (blinded) in P_{dep} – so a colluding depositor cannot substitute a different Identity, and the membership below is bound to the right point. (11 Python + 6 Solidity tests.)

==== Step B3: Submit `Notes.spendCoupledB1` ====

Carol calls the membership-bound B1 deposit:

```

notes.spendCoupledB1(
    proof, noteRoot, nullifier, face, recipient,
    issuer,                               // public issuer address
    eDepForIss,                           // (R_f, C_f) encrypting M_Carol
    b1Proof,                              // verifyDepositorBinding (incl. P_dep)
    membershipProof,                     // membership of M_Carol, bound to P_dep
);

```

The contract verifies the binding, then the membership of `b1Proof.P_dep` (the *same* point the binding committed), and emits `SpentCoupledB1(nullifier, face, recipient, issuer, eDepForIss)` with the ciphertext only the issuer can decrypt. Carol is now provably a registered Identity the issuer can name – the membership-bound dual of the A2 recipient receipt (`test/NotesCoupledB1.t.sol`, 6 tests).

==== Step B4: Issuer Decrypts and Produces a Receipt ====

The issuer scans `SpentCoupledB1` events, decrypts M_{Carol} with sk_{iss} , and *alone* produces a plaintext receipt naming both parties:

$$\mathcal{R}_{\text{iss}} = (M_{\text{issuer}}, M_{\text{Carol}}, v, E_{\text{dep} \rightarrow \text{iss}}, \text{vd}, \text{chainid}).$$

(`make_issuer_receipt` in `alberta_buck.wallet.b1_binding`.)

11.6 B1 in Code: The Issuer Names the Depositor

The same session, continued. Carol is a registered Identity the (public) issuer has never met; she holds a bearer note's opening. At spend she builds `eDepForIss` – her own M_{dep} encrypted under the issuer's key – and the five-relation depositor-binding sigma couples it to her payout account and to the blinded membership commitment P_{dep} :

```

from alberta_buck.wallet.b1_binding import (
    b1_bind_prove, b1_bind_verify, make_issuer_receipt, verify_issuer_receipt,
)

m_carol = rand_scalar(rng)
carol = account(m_carol)

```

```

tree.insert(carol["M"])

pk_iss = issuer["pk"]
binding, eDepForIss = b1_bind_prove(m_carol, carol["sk"], carol["E"], pk_iss,
                                   account=0xCA801, chainid=CHAINID, rng=rng)
ok = b1_bind_verify(carol["pk"], carol["E"], pk_iss, eDepForIss, binding,
                   account=0xCA801, chainid=CHAINID)
print(f"B1 depositor binding from Carol: verifies = {ok}")
print(f"  eDepForIss = ElGamal(M_carol under pk_iss)  R = {pt(eDepForIss.R)}")
print(f"  P_dep      = {pt(binding.P_dep)} (blinded; membership binds it)")

B1 depositor binding from Carol: verifies = True
  eDepForIss = ElGamal(M_carol under pk_iss)  R = (71317108..., 92478725...)
  P_dep      = (15608053..., 69485306...) (blinded; membership binds it)

```

The issuer alone – reading `eDepForIss` from the `SpentCoupledB1` event – decrypts and *proves* the depositor’s Identity, the dual of the A2 unilateral receipt. And the collusion-substitution is rejected: a ciphertext under any other key breaks the coupled sigma, so the depositor cannot misname itself.

```

rcpt_iss = make_issuer_receipt(issuer["sk"], issuer["M"], eDepForIss,
                              value=25, issuer=0xA11CE, chainid=CHAINID,
                              tree=tree, rng=rng)
res_b1 = verify_issuer_receipt(rcpt_iss, tree.root(), tree)
assert res_b1.valid
assert eq(res_b1.recipient_M, carol["M"])
print(f"issuer-unilateral receipt: {res_b1.reason}")
print(f"  names depositor {pt(res_b1.recipient_M)} == M_carol")

eDep_sub = elgamal_encrypt(carol["M"], mul(G1, rand_scalar(rng)), rand_scalar(rng))
ok_sub = b1_bind_verify(carol["pk"], carol["E"], pk_iss, eDep_sub, binding,
                      account=0xCA801, chainid=CHAINID)
print(f"substituted eDepForIss verifies = {ok_sub} (collusion-substitution rejected)")

issuer-unilateral receipt: VALID
  names depositor (55296567..., 79403012...) == M_carol
substituted eDepForIss verifies = False (collusion-substitution rejected)

```

11.7 Summary: What the Observer Sees (Mallory)

In all three flows the chain reveals:

- A fresh nullifier and a spent face amount.
- The payout address.
- For A1/A2: the coupled ciphertext $eIss$ and the point commitment P_I (both semantically secure; for A1 P_I commits the recipient, for A2 the private issuer).
- For B1: the issuer address and $E_{\text{dep} \rightarrow \text{iss}}$ (IND-CPA).
- The on-chain `identityRoot` (a single uint256, updated on registration, revealing nothing about which identities are active).
- *No* plaintext Identities, *no* link between the spend and any specific commitment in the tree, and *no* link between mint and spend events.

12 Demurrage-Carrying Transfers

The Note-spend payout in Step 5 went out as a plain `buck.transfer` that BUCK routed through its internal carrying path because the `Notes` pool is bound `isCarrying`. This section unpacks why a single per-account flag – not a separate method, not a marker interface – is sufficient, and what its ledger effect looks like.

12.1 Two Ways to Handle Demurrage at Transfer Time

BUCK demurrage is a balance-time integral: each account stores its `buckSeconds` (the crystallised $\int B dt$) plus the `timestamp` of its last touch, and its accrued fee is `rate \cdot (buckSeconds + balance \cdot elapsed)`. Two ledger responses to that accrued fee are distinguishable at transfer time, selected by the *sender's* `isCarrying` flag:

- *Deducting transfer (sender is non-Carrying, the default)*. On a 1,000-BUCK outgoing, Alice's accrued fee for the pre-transfer balance is settled at Alice – one-shot burned against her balance, and her `buckSeconds` is re-crystallised from the new balance. The 1,000 BUCK then lands at Bob *fresh*: his inflow begins accruing age from now, diluting his effective age. From a system perspective, accrued fees are burned at each transfer and the recipient enters fresh.
- *Carrying transfer (sender is isCarrying)*. On the same 1,000-BUCK outgoing, nothing is burned at the sender: the pool sheds a proportional slice of its live buck-seconds (`carried = liveBs \cdot 1000 / poolRaw`) and keeps its per-BUCK age. Bob inherits that slice (`buckSeconds + carried =`), so he absorbs the age of the BUCKs he just received. Nominal balances move by exactly 1,000 on each side; no fee is burned; total system BUCK-age is preserved.

The invariant the carrying variant keeps is: **nominal BUCK out = nominal BUCK in** (no silent burn at the transfer site), with the recipient's forward index shift representing the demurrage they will pay on their next standard outgoing transfer.

12.2 Why the Note Pool Needs It

Three structural features of the pool make the deducting default wrong:

1. **Per-note age opacity.** When Alice mints six notes, the pool records only six opaque Poseidon commitments. The pool cannot deduct "the demurrage *Alice's specific note* owes" at spend time, because the chain does not distinguish one commitment's age from another. Any per-note fee would have to be computed off the pool aggregate, which would force Bob to pay fees that the pool's other depositors owed – and, worse, differentiate one commitment's payout from another, destroying the anonymity set.
2. **Face-to-payout identity.** `face` is the public SNARK output and the value the mint circuit committed to. A deducting pool transfer would deliver `face - fee` to Bob while the proof claimed `face`. This would break the mint-to-spend value-conservation invariant: the SNARK's promise would not match the ERC-20 ledger movement, and the spend circuit would have to become aware of demurrage accounting to compensate.
3. **Pool as bookkeeping buffer.** The pool is not a long-term BUCK holder; every BUCK that enters is earmarked for one future redeemer. Deducting fees at the pool's egress would siphon

value into a burn that the pool has no way to replenish, producing a cumulative insolvency as outstanding notes circulate.

The carrying path fixes all three: no per-note age, **face** to payout, no pool insolvency.

12.3 Shipped Surface in Buck.sol

There is no separate **transferCarrying** method and no **TransferCarrying** event. The carrying behaviour is selected by a per-account flag, **isCarrying**, set when the account is bound:

```
// IdentityRegistry: governance sets isCarrying at bind time.
function bindContract(address t, ..., bool isPublicIdentity_, bool isCarrying_) external;
mapping(address => bool) public isCarrying;
```

The standard **transfer** path then dispatches on the *sender's* flag:

```
if (identity.isCarrying(from)) _carryingTransfer(from, to, amount);
else _nonCarryingTransfer(from, to, amount);
emit Transfer(from, to, amount); // one standard event either way
```

The preconditions are exactly those of any transfer (sender and recipient both **isVerified**, the bilateral receipt-fragment / **_identityHash** identity check), because it *is* an ordinary transfer. **_carryingTransfer** sheds a proportional slice of the sender's live buck-seconds onto the recipient rather than burning a fee at the sender:

```
liveBs = from.buckSeconds + from.raw * elapsed;
carried = liveBs * amount / from.raw;
from.buckSeconds = liveBs - carried; // sender keeps its per-BUCK age
recipient.buckSeconds += carried; // recipient inherits the age
```

Carrying accounts hold no NFT-backed credit and cannot go negative (the path asserts **amount** < **raw**). What makes an account pool-friendly is the *flag*, set once at bind time; there is no per-call method to invoke and no balance precondition beyond that non-negative assertion.

12.4 Who Else Is Carrying

The Note pool is the canonical **isCarrying** account, but the flag is not pool-private: governance binds AMM pools, the Jubilee, and the Notes pool **isCarrying**, and could bind any transitory-holder contract the same way – escrow accounts whose beneficiaries are the true owners, payroll contracts pulling from an employer, cross-chain bridge egresses. The semantic invariant is "the account is a transitory holder of BUCK earmarked for its recipients," and in that case carrying the age forward matches the economic reality.

For ordinary peer-to-peer accounts the deducting default remains correct: the sender is paying from their own pocket and is the natural place to burn their own accrued fee, so EOAs are bound non-Carrying.

12.5 Open Question: Identity Receipts on Carry

Because a carrying transfer *is* an ordinary **transfer**, it uses the same identity predicate – both parties **isVerified**, with the receipt-fragment fallback to **_identityHash** on the public side when a party has **isPublicIdentity** = true=. The Notes pool side relies on its bound Public Identity to satisfy the bilateral check. Whether a carrying transfer between two Encrypted-Identity counterparties should require its own per-call Chaum-Pedersen receipt is an open question; for the Notes pool it is moot (the pool is bound under a Public Identity), but if transitory-holder carry is ever opened to private-to-private pairs an approve-style receipt may be wanted for auditability. See the **Open Work Items** in alberta-buck-ethereum.org.

13 Putting the Guarantees Side-by-Side

The mint walkthrough established five guarantees; the spend walkthrough adds four more that become concrete only once the redemption path exists.

Stage	Guarantee	Mechanism
Mint	Atomic conservation	Single EVM call: proof verify, <code>transferFrom</code> , append-cms all-or-nothing
Mint	Issuer-bound, recipient-blinded	<code>msg.sender</code> is public; <code>cm_i</code> is Poseidon-hidden over (<code>flavor</code> , <code>v</code> , <code>rho</code> , <code>idHash</code> , <code>pred</code>)
Mint	Per-note disclosure (recipient only)	Off-chain opening contains exactly one note's witness fields
Mint	Offline receipt reconstruction	<code>T_mint</code> calldata + opening + on-chain Identity lookup is sufficient and self-checkable
Mint	Adversarial-prover soundness	Groth16 knowledge soundness on fixed on-chain verifier; ERC-20 balance/allowance integrity
Spend	Redemption unlinkable to specific <code>cm</code>	<code>cm</code> is a <i>private</i> witness under public <code>noteRoot</code> ; only nullifier + face + recipient revealed
Spend	Double-spend prevented	Deterministic nullifier + one-bit <code>nullifiers</code> mapping; rejected on reuse
Spend	Recipient-bound payout	<code>recipient</code> is a public SNARK input; proof rejects if re-targeted in-flight
Spend	Value conservation across mint<->spend	Range-bounded mint <code>v[i]</code> + spend <code>face</code> == <code>v</code> ; cumulative payouts $\leq$ <code>totalFace</code> per batch

All nine rows are enforced by code in the tree. Mint landed in two phases: Phase 5 (`Notes.sol` with a stub verifier) and Phase 7 (per-leaf on-chain Poseidon insertion with `mint.circom`, `MintGroth16Verifier`, `MintVerifierAdapter`). Phase 7-bis moves the Merkle insertion *into* the SNARK (`mint_batch.circom`, per-N Groth16 verifiers, per-N adapter dispatch); the chain writes the SNARK-attested `newRoot` into the same 30-slot root- history ring `Notes.sol` already maintained for the spend side. Spend is unchanged by the pivot: `spend.circom` in bearer shape, `SpendGroth16Verifier`, `SpendVerifierAdapter`, the on-chain Merkle accumulator, and BUCK's `isCarrying` carrying-transfer path. The IdentityRegistry provides the issuer-attestation and recipient-verification legs (extended with the Identity-M deposit gates – `verifyDepositCoupling` for A1/A2, `verifyDepositorBinding` for B1, plus the G1-tie membership verifier), and the formal proofs (Theorems 7-11) provide the cryptographic backstop. Each spend's identity binding – an EIP-196 Okamoto sigma proving the spender is the named registered Identity, bound to a Poseidon-Merkle membership of the *same* committed point – costs ~36K gas for the sigma plus a constant ~350K-gas membership verify, much cheaper than proving native BN254 $\mathbb{G}_1$ equality inside the spend SNARK (~+250K R1CS).

Why the spend carries this apparatus at all – anonymity is the cost. A direct transfer reaches the same mutual-decryptability guarantee with no SNARK, because the bilateral `approve` names both parties at once. The Notes detour pays the mint SNARK, the recipient-key blinding, and the deposit-coupling purely to buy mint <-> spend unlinkability: the issuer's registered anchor lives at mint, the recipient's at spend, and they are never co-present, so the apparatus is what bridges them. Public-issuer notes (A1/B1) keep the cheap, fully-sound path; A2 is the premium that hides the issuer too. See *alberta-buck-notes* ("Why A2 Costs More: Anonymity Is the Cost") and *alberta-buck-notes-decryptability* ("The EOA Transfer Mirror").

14 Open Questions and the Identity-M Transition

The redemption walk above *is* the identity-M spend – the only spend path. The legacy account-pinned A-spend (`spend_a.circom`, `Notes.spendACP`, `verifySpendCP`) and the unbound generic `Notes.spend` have been **removed**; what remains is one gadget (`verifyDepositCoupling` / `verifyDepositorBinding`) + the G1-tie membership `identity_membership_g1tie.circom`, wired through `spendCoupled{A1,A2,B1}`, and all its load-bearing pieces are *shipped*:

1. **On-chain identity accumulator – DONE.** `IdentityRegistry.identityRoot` is an incremental Poseidon accumulator (depth 10, Tornado-style `filledSubtrees`) updated on every `register()` / `bindContract()`, with the *commit-before-use* discipline so a newly issued *M* is usable only once the root reflects it. The one canonical off-chain tree (`alberta_buck.registry.tree.Identity`

shared by the wallet and the sim) matches it byte-for-byte; the `CentralMerkleService` aggregator (49 tests) feeds sub-roots in.

2. **Non-native $\mathbb{G}_1$ tie in the membership circuit – DONE.** `identity_membership_g1tie.circom` proves the native Poseidon-Merkle membership *and* the $\mathbb{G}_1$ tie $P_I = M + bH$ in one Groth16 proof (one BN254 $\mathbb{G}_1$ addition via circom-lib), exported to `IdentityMembershipG1TieVerifier.sol` (on-chain verified, $\sim 350K$ gas).
3. **Route the spend through the Identity-M gates, with the binding – DONE.** `Notes.spendCoupledA2` calls `verifyDepositCoupling` and the membership verifier, **passing `dc.P_I` to both** so the two halves are bound to the same committed point (the verifier derives its public-input limbs from `dc.P_I`, not from the proof bytes – `IdentityMembershipBinding.t.sol`, `NotesCoupledA2.t.sol`). This is the step that closes the A2 recipient-key *collusion* gap: a bogus *eIss* makes P_I 's point a non-member, so no membership proof exists and the deposit reverts. A1 and B1 take the identical bound-membership path (`spendCoupledA1` / `spendCoupledB1`).
4. **Legacy retirement and the B1 counterpart – DONE.** The account-pinned A-spend (`spend_a.circom` + its verifier, `Notes.spendACP`, `verifySpendCP`, and the `spend_a` / `spend_cp` wallet modules) and the unbound generic `Notes.spend` are removed; only the flavor-agnostic `spend.circom` note proof survives, shared by all three coupled spends. The B1 depositor-side membership counterpart is implemented: `spendCoupledB1` binds `verifyDepositorBinding`'s committed point `P_dep` to the membership exactly as A2 binds `dc.P_I` (`NotesCoupledB1.t.sol`).
5. **The note $\leftrightarrow$ *eEnc* re-encryption tie – DONE.** The flavour-agnostic note proof exposes no *idHash*, so gates 1–4 alone verified a ciphertext *of the depositor's choosing* – leaving addressed-binding ("only M_{rec} can spend") and the A2 substitution escape ("attach a different, nameable *eEnc*") unenforced. `circuits/note_binding.circom` ($\sim 2.4M$ non-linear constraints; the ElGamal-structure optimisation keeps every EC operation fixed-base) proves the spend's *eEnc* re-encrypts the ciphertext committed in the *idHash* behind *this spend's nullifier*, and that its decryption is the sigma's own committed P_I point. `NoteBindingVerifierAdapter` derives all 25 public inputs on chain from the caller's `nullifier` / `eEnc` / `dc.P_I`; the governance slot `Notes.setNoteBindingVerifier` rotates the verifier. (*Proofs*, Theorem 12; toolchain history in *alberta-buck-verifier-implementation*.)

With items 1–5 shipped the transition is **complete**: every mint and spend is Identity-M-bound, the on-chain `identityRoot` gates every spend, and the note's counterparty Identity is always recoverable and – because the membership is *bound* to the sigma's committed point, and the verified ciphertext is *bound to the spent note* – always nameable, even under collusion and ciphertext substitution. The only remaining pre-deployment item is a proper Powers-of-Tau ceremony (the dev ptaus, including the `pot22` behind the note-binding verifier, use dev-only entropy).

Additional items tracked across the series:

- **Mint-side deferred-approve handshake (shipped).** The public-issuer Schnorr binding (`verifyIssuerSchnorr`) and the private-issuer re-encryption binding (`verifyIssuerReenc`) are deployed and tested. Every mint carries a verified issuer Identity binding.
- **Variable-N batch size (resolved via pinned set).** The wallet rounds any requested split up to the smallest pinned $N \geq N_{\text{requested}}$ from the deployed set (recommended $\{16, 128, 1024\}$), padding with $v = 0$ dummy openings. Each pinned N has its own verifier contract; `MintVerifierAdapter` dispatches on `cms.length`.

- **Prover contention (resolved operationally, not protocolly).** The batch- mint pivot makes each mint a rollup-style transaction with a state-state guard on `oldRoot` and `nextLeafIndex`. Losers in a race re-prove; no BUCK is lost. Initial deployment uses wallet-side optimistic retry; a sequencer/aggregator is deferred until real-world contention warrants it.

15 Privacy Audit: Data Revealed Per Step

The whole construction threads one needle: publish enough that each party can *name* the other under compulsion, while publishing so little that no *third party* can name either, or link payments in bulk. This section consolidates the per-step disclosure analysis for the shipped Identity-M surface. (It originated as the gap analysis in *alberta-buck-notes-decryptability*; the gaps it tracked are closed, and this is the surviving reference form.)

15.1 Requirements and adversaries

Three properties, against three adversaries:

- **(Bulk-tracing resistance.)** *Mallory*, a passive observer with the full chain history and unbounded compute but no party's secrets, cannot reconstruct who paid whom, how much, or link a mint to its spend(s).
- **(Third-party non-disclosure.)** Nothing on chain reveals either party's KYC-bound Identity point *M* to a non-participant – not even to the *counterparty's* counterparty, and not in aggregate across many payments.
- **(Mutual decryptability.)** Each of the two participants, from data they hold plus public chain state, can produce a sound receipt naming the other's registered *M* – and a *colluding* pair cannot arrange a payment for which no such receipt exists (the non-deniable-receipt invariant).

The first two are *confidentiality* against outsiders; the third is *accountability* between insiders. They pull in opposite directions, so every published datum below is justified against all three.

15.2 The on-chain footprint of each step

Step	Published on chain
<code>register / bindContract</code>	<code>addr, pk, E_addr</code> (ElGamal ct of <i>M</i> under own <i>pk</i>), <code>isPublicIdentity</code> , <code>identityRoot</code>
<code>mint</code> (any flavor)	<code>Minted(issuer, totalFace, startIndex, count, newRoot)</code> ; <code>cms[]</code> in <code>calldata</code> (opaque Poseidon)
<code>mint public</code> (A1/B1)	+ <code>IssuerBound(issuer, newRoot)</code> ; one Schnorr over <code>keccak(cms)</code>
<code>mint private</code> (A2)	+ <code>IssuerReencBound(issuer, newRoot, count)</code> ; per-leaf blinded binding ($Q, U, \hat{T}, A_1..A_5$)
A1/A2 spend	<code>SpentCoupledA1/A2(nf, face, recipient, P_I)</code> ; <i>eEnc</i> (fresh re-encryption), the coupling sigma, the membership
B1 spend	<code>SpentCoupledB1(nf, face, recipient, P_{dep})</code> ; <i>eDepForIss</i> , the depositor-binding sigma, the membership proof

Two facts do the heavy lifting throughout. First, `cms[]` are Poseidon commitments: a leaf reveals *nothing* about (*flavor, v, ρ , idHash, predicate*) without ρ . Second, the spend nullifier $nf = \text{Poseidon}_3(\rho, \text{idHash}, \text{tag})$ is a PRF output over secret material, so it is unlinkable to any `cm` without the opening (*Proofs*, Theorem 10).

15.3 Why bulk-tracing and third-party privacy hold

Mint is opaque; spend is unlinkable to mint. At mint Mallory sees opaque `cms[]` and a `totalFace`; she cannot read any v_i , recipient, or flavor (Poseidon hiding). At spend she sees a fresh nf , a `face`, and a `recipient`; nf is a PRF over $(\rho, idHash)$, so inverting it to a specific `cm` is pre-image resistance. No field of a spend points back to a mint: the spend's $eEnc$ is a *fresh re-encryption* of the leaf ciphertext (uniformly re-randomised; the note-binding relation deliberately proves re-encryption *equivalence*, never equality with the public mint-time $eIss$). The only residual linkage surfaces are the two that *must* be public for the ERC-20 to pay out – `face` and `recipient` – mitigated by fixed denominations and relayed payouts, orthogonal to this document.

Identities never appear on chain – the load-bearing check. A registered E_{addr} is an ElGamal *ciphertext* of M ; reading it yields nothing without sk . The danger is in the *bindings*, which manipulate M and the recipient point algebraically. Concretely, the A2 mint binding must publish enough to prove " $eIss$ re-encrypts the issuer's registered M_{iss} under the recipient's point" without revealing either. This is exactly where the first draft failed – the canonical near-miss, kept here because it is the clearest way to see the invariant:

```
Near-miss (rejected draft). The binding published T = r' * pk_rec.
But the leaf is C_i = M_iss + r' * pk_rec, so ANY observer computes
    M_iss = C_i - T.
```

At mint `msg.sender` IS the issuer, so this publicly bound the issuer's address to its Identity -- collapsing A2 to A1 and breaking BOTH third-party non-disclosure and bulk unlinkability (every A2 mint by the same issuer now carries the same recoverable `M_iss`).

The shipped binding blinds it: it publishes $\hat{T} = r'pk_{rec} + \gamma G$ for a fresh γ . Now $C_i - \hat{T} = M_{iss} - \gamma G$, a uniformly random offset from M_{iss} – so $\hat{T}$ reveals nothing. The recipient's point is likewise hidden (inside the perfectly-hiding Q and the uniform $U, \hat{T}$), and soundness is undisturbed because γ cancels in the verification relations. An A2 mint's on-chain footprint is, to Mallory, a vector of uniformly-distributed group elements.

The spend-side sigmas reveal the spender's action, not any Identity. The deposit-coupling and depositor-binding sigmas are HVZK; their only Identity-derived public values are the blinded commitments $P_I = M_I + bH$ / $P_{dep} = M_{dep} + bH$, perfectly hiding for uniform b . The membership and note-binding Groth16 proofs are zero-knowledge; their public inputs are the nullifier (PRF), $eEnc$ (uniform), and P (hiding). `msg.sender` is visible – exactly as the sender of any ERC-20 transfer is – but no *Identity* is.

15.4 Why mutual decryptability holds – the final data set

For each flavor, the data each party ends up holding, and the predicate it satisfies (the receipt verifiers in `alberta_buck.wallet` are the public predicate `RcptVerify`; see the executable sections above):

Flavor	Recipient names issuer via	Issuer names depositor via
B1	in-clear M_{iss} + batch Schnorr (<code>verifyIssuerSchnorr</code>)	<code>SpentCoupledB1</code> : decrypt $eDepFo$
A1	in-clear M_{iss} + batch Schnorr	chose M_{rec} at mint (symmetric k)
A2	decrypt $eIss$ under m_{rec} ; binding + membership prove it is the registered minter (A2 in Code)	chose M_{rec} at mint

The A2 row is the subtle one. The recipient's receipt composes three facts: the mint anti-framing binding (the ciphertext re-encrypts the issuer's *registered* record), their own verifiable decryption

(it opens under m_{rec} to the named point), and the membership of that point in rt_{id} . No issuer cooperation is needed – the receipt is *unilateral* – and no new proof is needed at receipt time: the coupling falls out of composing the mint binding with the recipient’s own decryption.

15.5 The colluding pair, and where each escape is closed

Completeness (an honest recipient *can* produce a receipt) is met off chain. The third clause – no deniability even when payer and payee *collude* – needs the binding to be *unavoidable*. Three escapes existed on the road here, each now closed at a specific gate:

1. *A binding-less leaf.* Closed at **mint**: **Notes.mint** requires a verified issuer binding (Schnorr or re-encryption) for every leaf; the SNARK’s per-leaf **issuerMode** signal makes a private-issuer bearer leaf unprovable (no B2 codepoint).
2. *A nameable note deposited by the wrong Identity.* Closed at **spend** by the coupling sigma + bound membership: the depositing account must hold the Identity scalar that decrypts the verified ciphertext.
3. *An un-nameable note made spendable by ciphertext substitution.* Closed at **spend** by the note-binding tie: the verified *eEnc* must re-encrypt the ciphertext committed behind *this spend’s nullifier* (*Proofs*, Theorem 12) – demonstrated executably in the A2 in Code counter-example above.

DRAFT

16 Appendix: The Hostile-Issuer Mint Ledger

The batch-mint design memo (*alberta-buck-notes-rollup-mint*) audited every attack vector available to a hostile minter under the Phase-7-bis pivot (duplicate-cm detection moved from an on-chain check to economic self-enforcement). The ledger survives as the soundness reference for the shipped mint path:

- **A1. Underpay totalFace.** totalFace is a public input bound by the sum constraint and Num2Bits(128); transferFrom pulls exactly totalFace. *Intact.*
- **A2. Field-overflow on sum.** Num2Bits(128) on each v_i and on totalFace leaves 62-bit margin against field wrap at $N \leq 2^{64}$. *Intact.*
- **A3. Duplicate cm across mints.** Insertion succeeds at a new leaf; the nullifier is deterministic in the opening, so exactly one copy is spendable. The duplicate-minter paid twice for one spend; no one else is harmed. *Economically self-enforcing.*
- **A4. cm collision with ZERO_VALUE or 0.** Domain-separated keccak vs. the Poseidon-5 commitment space: 2^{-254} . *Intact.*
- **A5. Lying about oldRoot.** Contract asserts equality with the recorded root; revert. *Intact.*
- **A6. Lying about newRoot.** The SNARK derives newRoot from oldRoot plus the folded leaves; unsatisfiable. *Intact.*
- **A7. Front-running to invalidate a pending mint.** The loser's oldRoot check fails before any BUCK moves; re-prove and retry. *No value loss; operational concern only (see Prover Contention).*
- **A8. Mint with an idHash naming someone else.** The mint-side issuer bindings (Schnorr for public, verifyIssuerReenc for private) force the identity payload over the minter's own registered record – the anti-framing half of Theorem 7. A self-harming junk idHash remains possible and remains the minter's own loss. *Intact.*
- **A9. Mint junk leaves with unknown openings.** The circuit requires knowledge of every opening; junk is a paid donation to the pool. *Intact.*
- **A10. Spend nullifier reuse.** nullifiers[nf] set-once; second spend reverts. *Intact.*
- **A11. Spend front-running.** recipient is bound in the spend SNARK's public inputs; a proof cannot be replayed toward a different recipient. *Intact.*
- **A12. Lying about nextLeafIndex.** Asserted against on-chain state and consumed by the SNARK's insertion paths; either reverts or produces an unhonoured root. *Intact.*

Headline: every hostile-minter guarantee holds under the batch-mint pivot. Buggy *wallets* (e.g., a ρ RNG collision) are no longer caught by the chain, but the failure mode is "wallet owner forfeits one note", not "pool drains".

17 Security Properties and the Tests That Witness Them

Every property claimed in this document is pinned by an executable witness – the Python suites exercise the wallet reference, the Forge suites the on-chain verifiers, and the cross-artifact vectors (`alberta_buck.wallet` $\rightarrow$ `vm.readFile`) keep the two byte-for-byte aligned.

Suite	Count	What it demonstrates
<code>alberta_buck/test/test_unilateral_a2.py</code>	11	A2 mint binding; delivery decrypt; deposit coupling (any account); unilateral receipt n
<code>alberta_buck/test/test_unilateral_a1.py</code>	7	A1 mint; <code>idHash</code> binds the public issuer + Schnorr; recipient-side receipt names both; t
<code>alberta_buck/test/test_b1_binding.py</code>	11	B1 depositor binding; issuer-unilateral receipt names the depositor; collusion-substituti
<code>test/UnilateralA2.t.sol</code>	7	on-chain <code>verifyDepositCoupling</code> parity + soundness (tampered response / P_I / cipher
<code>test/B1Binding.t.sol</code>	6	on-chain <code>verifyDepositorBinding</code> parity + soundness
<code>test/IdentityMembershipBinding.t.sol</code>	5	the P binding: bound point verifies; mismatched P_x/P_y /root rejected
<code>test/NotesCoupledA1.t.sol</code>	8	end-to-end <code>spendCoupledA1</code> : completeness + payout; tamper/double-spend reverts
<code>test/NotesCoupledA2.t.sol</code>	10	end-to-end <code>spendCoupledA2</code> : completeness + payout + <code>SpentCoupledA2</code> ; bad coupling /
<code>test/NotesCoupledB1.t.sol</code>	6	end-to-end <code>spendCoupledB1</code> : the depositor-binding + bound membership path
<code>test/NoteBindingVerifier.t.sol</code>	9	the note $\leftrightarrow$ <code>eEnc</code> tie: real Groth16 proof verifies (raw + adapter); tampered nullifier /

Build/test entry points: `make nix-test` (Forge), `make nix-test-python` (wallet reference),
`make nix-snark-note-binding` (note-binding circuit + trusted setup + vectors).

18 Cross-References

- alberta-buck-notes.org – architectural overview of the Notes primitive (flavors, Merkle tree, demurrage interaction). Identity-M edition. The other living half of the Notes documentation pair (this document is the worked, executable half); its *Document Series* section is the canonical map of which satellite memos are living, forward designs, or historical.
- alberta-buck-proofs.org – formal correctness arguments (Theorems 7-12; Open Questions in Part IV).
- alberta-buck-verifier-implementation.org – SNARK toolchain engineering: the witness-generation root cause, rapidsnark integration, trusted setups.
- alberta-buck-ethereum.org – Solidity surface, gas costs, phase-by-phase implementation status, Open Work Items.
- alberta-buck-identity.org – IdentityRegistry, PS credentials, and the bilateral ElGamal handshake that anchors `msg.sender` to a verifiable identity.
- alberta-buck-notes-identity-axis.org – *historical*: the unified identity-M design memo; absorbed into this document and the notes overview.
- alberta-buck-notes-unilateral.org – *historical*: the A2 corner as first built; absorbed into "A2 Identity-Targeted Spend" / "A2 in Code" above.
- alberta-buck-notes-decryptability.org – *historical*: the non-deniable-receipt gap analysis and fix design; the invariant lives in the notes overview, the privacy audit above, the theorems in Proofs, and the fix is implemented (`note_binding.circom`).
- alberta-buck-notes-rollup-mint.org – *historical*: Phase-7-bis batch-mint design memo (R1CS counts per N, gas projections, concurrency model); its hostile-issuer audit survives as the appendix above.
- `circuits/mint_batch.circom` – the Phase-7-bis batch-mint circuit (one compile per pinned N).
- `circuits/spend.circom` – the Phase-7 spend circuit (bearer-shape).
- `circuits/note_binding.circom` – the note $\leftrightarrow eEnc$ re-encryption tie (A1/A2 spends).
- `src/Notes.sol`, `src/MintGroth16Verifier*.sol`, `src/MintVerifierAdapter.sol`, `src/SpendGroth16Verifier.sol`, `src/SpendVerifierAdapter.sol`, `src/NoteBindingGroth16Verifier.sol`, `src/NoteBindingVerifierAdapter.sol` – the on-chain code paths walked above.
- `scripts/snark/prove_mint_batch.js`, `scripts/snark/prove_spend.js` – the reference provers used in tests; `scripts/snark/setup_note_binding.sh` – the note-binding circuit + trusted-setup pipeline.
- `test/MintVerifier.t.sol`, `test/SpendVerifier.t.sol`, `test/NoteBindingVerifier.t.sol` – the Foundry tests that pin the soundness stories to executable assertions.